



Security Review For NestUSD



Collaborative Audit Prepared For: **NestUSD**
Lead Security Expert(s): **Oxeix**

Date Audited: **no**
July 6 - July 16, 2026

Introduction

Nest – Where stocks become stable dollars.

Nest is a Solana stablecoin protocol that turns tokenized equities into onchain dollar liquidity. Users can deposit tokenized stocks as collateral to borrow nUSD, convert between USDC and nUSD through the protocol's peg stability module, and stake nUSD for snUSD to earn yield -- all without selling their underlying equity exposure.

Scope

Repository: NestUSD/contracts

Audited Commit: fb358f3ae0a5122e0459ee16e09ec5edd898c218

Final Commit: 77eee01b550575e229d9b085551d3d854a5ee52e

Files:

- crates/nest-domain/src/cdp.rs
- crates/nest-domain/src/constants.rs
- crates/nest-domain/src/error.rs
- crates/nest-domain/src/lib.rs
- crates/nest-domain/src/math.rs
- crates/nest-domain/src/oracle.rs
- crates/nest-domain/src/psm.rs
- crates/nest-domain/src/staking.rs
- programs/nest-core/src/accounts/admin.rs
- programs/nest-core/src/accounts/buyback.rs
- programs/nest-core/src/accounts/cdp.rs
- programs/nest-core/src/accounts/kamino.rs
- programs/nest-core/src/accounts/liquidation.rs
- programs/nest-core/src/accounts/oracle.rs
- programs/nest-core/src/accounts/psm.rs
- programs/nest-core/src/accounts/revenue.rs
- programs/nest-core/src/accounts.rs
- programs/nest-core/src/accounts/setup.rs
- programs/nest-core/src/errors.rs
- programs/nest-core/src/helpers/auth.rs

- `programs/nest-core/src/helpers/domain.rs`
- `programs/nest-core/src/helpers/errors.rs`
- `programs/nest-core/src/helpers/kamino.rs`
- `programs/nest-core/src/helpers/oracle.rs`
- `programs/nest-core/src/helpers/psm.rs`
- `programs/nest-core/src/helpers/revenue.rs`
- `programs/nest-core/src/helpers/staking_reader.rs`
- `programs/nest-core/src/helpers/token.rs`
- `programs/nest-core/src/ix/admin.rs`
- `programs/nest-core/src/ix/cdp.rs`
- `programs/nest-core/src/ix/kamino.rs`
- `programs/nest-core/src/ix/liquidation/buyback.rs`
- `programs/nest-core/src/ix/liquidation/instant.rs`
- `programs/nest-core/src/ix/liquidation/insurance.rs`
- `programs/nest-core/src/ix/liquidation.rs`
- `programs/nest-core/src/ix/liquidation/two_step.rs`
- `programs/nest-core/src/ix/mod.rs`
- `programs/nest-core/src/ix/oracle.rs`
- `programs/nest-core/src/ix/psm.rs`
- `programs/nest-core/src/ix/revenue.rs`
- `programs/nest-core/src/ix/setup.rs`
- `programs/nest-core/src/lib.rs`
- `programs/nest-core/src/params.rs`
- `programs/nest-core/src/state.rs`
- `programs/nest-stake/src/accounts.rs`
- `programs/nest-stake/src/errors.rs`
- `programs/nest-stake/src/helpers.rs`
- `programs/nest-stake/src/instructions.rs`
- `programs/nest-stake/src/lib.rs`
- `programs/nest-stake/src/params.rs`
- `programs/nest-stake/src/state.rs`

Final Commit Hash

77eee01b550575e229d9b085551d3d854a5ee52e

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
8	10	5

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue H-1: Unharvested revenue can be stolen by last-minute stakers [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/issues/52>

Summary

Currently, the staking share price doesn't account for nUSD revenue that is already available in `revenue_nusd_vault` but has not yet been harvested. A user can stake immediately before `harvest()` is called and receive shares priced as if the pending revenue doesn't exist, then benefit from that old revenue once it is harvested.

Vulnerability Detail

When a user stakes, the amount of shares minted is calculated from `staking_vault_nusd` only. The calculation excludes harvestable revenue sitting in `revenue_nusd_vault`:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-stake/src/instructions.rs#L66-68>

```
let mut pool = domain_pool(&ctx.accounts.staking_state);
let now = Clock::get()?.unix_timestamp;
let minted_shares = pool.stake(amount as u128, now).map_err(map_stake_error)?;
```

The domain logic calculates entry assets only from the staking vault and reserved claims:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/crates/nest-domain/src/staking.rs#L47-49>

```
pub fn stake_entry_assets(&self) -> Result<u128> {
    checked_sub(self.staking_vault_nusd, self.reserved_pending_claims)
}
```

And that value is used as the denominator for share minting:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/crates/nest-domain/src/staking.rs#L75-88>

```
pub fn stake(&mut self, amount: u128, now: i64) -> Result<u128> {
    if amount == 0 {
        return Err(NestError::InvalidParameter);
    }
    self.sync_vesting(now)?;
    let entry_assets = self.stake_entry_assets()?;
    let shares = if self.total_shares == 0 {
        amount
    } else {
```

```
    if entry_assets == 0 {  
        return Err(NestError::InvalidParameter);  
    }  
    mul_div_down(amount, self.total_shares, entry_assets)?  
};
```

So the share price denominator is effectively $\text{staking_vault_nUSD} - \text{reserved_pending_claims}$ and it doesn't include $\text{revenue_nUSD_vault.amount} - \text{revenue_baseline_nUSD}$. Later, `harvest()` transfers that revenue into the staking vault and increases `staking_vault_nUSD`, making the newly minted shares entitled to a portion of revenue that was generated before the user joined. So let's consider the scenario with 2 users for simplicity:

User1 owns 100 shares backed by 100 nUSD. Now 100 nUSD of revenue is sent to `revenue_nUSD_vault`. User2 stakes 100 nUSD before anyone calls `harvest()`. User2 receives 100 shares because the share price ignores the pending 100 nUSD revenue and then calls `harvest()` with that amount. So now the staking vault now contains 300 nUSD backing 200 shares. User2 owns 50% of the pool and can eventually withdraw 150 nUSD, capturing 50 nUSD of user1 previously accrued revenue.

Impact

Existing stakers can lose a portion of already-earned but unharvested revenue to new stakers.

Tool Used

Manual Review

Recommendation

Before minting new staking shares, account for pending harvestable revenue.

Discussion

ducksquadd

<https://github.com/NestUSD/contracts/commit/2e4b8666e4b06bf960545846f62af51a954ef9ef>

0502lian

fixed

Issue H-2: Stale staker revenue liability can brick core operations and lock the funds [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/issues/53>

Summary

`protocol.realized_revenue_for_stakers` is increased when core routes revenue to stakers, but it is never reduced after that revenue is harvested and withdrawn by stakers. As a result, the core program can keep enforcing coverage for revenue that has already been paid out, causing future repayments, liquidations and other operations to fail.

Vulnerability Detail

When revenue is allocated to stakers, core increments `realized_revenue_for_stakers`:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/helpers/revenue.rs#L97-100>

```
protocol.realized_revenue_for_stakers = protocol
    .realized_revenue_for_stakers
    .checked_add(staker_delta)
    .ok_or(error!(CoreError::MathOverflow))?;
```

Core later checks that the recorded staker revenue is still backed by either unharvested nUSD sitting in the staker revenue vault or assets currently held in the staking vault:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/helpers/token.rs#L133-142>

```
fn require_recorded_staker_revenue_covered(
    unharvested_revenue: u64,
    staking_assets: u128,
    recorded_revenue: u128,
) -> Result<()> {
    let covered_assets = (unharvested_revenue as u128)
        .checked_add(staking_assets)
        .ok_or(error!(CoreError::MathOverflow))?;

    require!(covered_assets >= recorded_revenue, CoreError::InvalidParameter);
    Ok(())
}
```

However, after stakers harvest the revenue and later complete unstaking, the staking program decreases `staking_vault_nusd` and transfers nUSD out to the user:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-stake/src/instructions.rs#L245-263>

```
let assets = pool
    .complete_unstake(
        domain::PendingWithdrawal {
            shares: ctx.accounts.pending_withdrawal.shares,
            request_ts: ctx.accounts.pending_withdrawal.request_ts,
        },
        now,
    )
    .map_err(map_stake_error)?;

apply_pool(&mut ctx.accounts.staking_state, pool);
```

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/crates/nest-domain/src/staking.rs#L151-152>

```
self.total_shares = checked_sub(self.total_shares, pending.shares)?;
self.staking_vault_nusd = checked_sub(self.staking_vault_nusd, assets)?;
```

There is no corresponding update to `protocol.realized_revenue_for_stakers`. The core-side value remains permanently high even though the revenue has already been paid out.

Impact

Once cumulative revenue paid out exceeds current backing, all the core operations are technically bricked, including the repayments and liquidations. As a result, borrowers then can't repay or reclaim collateral.

Tool Used

Manual Review

Recommendation

Consider not treating `realized_revenue_for_stakers` as a permanent cumulative value in coverage checks. Track only outstanding unpaid staker revenue, and reduce that liability once revenue is harvested or paid out.

Discussion

ducksquadd

<https://github.com/NestUSD/contracts/commit/85aa0106a2e0a9f90996cf3bf71457155884070f>

0502lian

Fixed in <https://github.com/NestUSD/contracts/commit/ba4212595e23144b28e8964bcdebbbfde7340d2>

Issue H-3: Permissionless dust harvest can indefinitely postpone pending unstake completion [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/issues/54>

Summary

`request_unstake` burns the user's `snUSD` and creates a pending withdrawal that has no cancel or rollback path. However, `complete_unstake` refuses to redeem while any staking revenue is still vesting. Since `harvest` is permissionless and accepts any positive amount, an attacker can repeatedly transfer 1 base unit of `nUSD` into the staking revenue vault and call `harvest(1)` to restart a fresh 7 day vesting window before pending withdrawals can complete.

Vulnerability Detail

When a user requests to unstake, the staking program records a pending withdrawal and then burns the user's `snUSD`:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-stake/src/instructions.rs#L195-L234>

```
let pending = pool
    .request_unstake(shares_u128, now)
    .map_err(map_stake_error)?;
apply_pool(&mut ctx.accounts.staking_state, pool);
ctx.accounts.staking_state.total_user_shares = ctx
    .accounts
    .staking_state
    .total_user_shares
    .checked_sub(pending.shares)
    .ok_or(error!(StakeError::MathOverflow))?;

let account = &mut ctx.accounts.pending_withdrawal;
account.owner = ctx.accounts.owner.key();
account.staking_state = ctx.accounts.staking_state.key();
account.shares = pending.shares;
account.request_ts = pending.request_ts;
account.completed = false;
account.bump = ctx.bumps.pending_withdrawal;

token_burn_checked(
    ctx.accounts.snusd_token_program.to_account_info(),
    ctx.accounts.snusd_mint.to_account_info(),
    ctx.accounts.owner_snusd_account.to_account_info(),
    ctx.accounts.owner.to_account_info(),
```

```

    shares,
    ctx.accounts.snusd_mint.decimals,
    &[],
)?;

```

The pending withdrawal PDA is unique for the owner and staking state, so the same user cannot create another pending withdrawal for the same staking state while this one is stuck:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-stake/src/accounts.rs#L85-L96>

```

#[account(init, payer = owner, space = 8 + PendingWithdrawalAccount::INIT_SPACE,
↳ seeds = [b"pending", owner.key().as_ref(), staking_state.key().as_ref()], bump)]
pub pending_withdrawal: Box<Account<'info, PendingWithdrawalAccount>>,

```

To finish the withdrawal, the domain logic first checks the cooldown and then synchronizes vesting. Even after the user's cooldown has elapsed, it rejects completion if any unvested revenue remains:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/crates/nest-domain/src/staking.rs#L131-L148>

```

pub fn complete_unstake(&mut self, pending: PendingWithdrawal, now: i64) ->
↳ Result<u128> {
    require_nonnegative_ts(now)?;
    require_nonnegative_ts(pending.request_ts)?;
    require_nonnegative_ts(self.cooldown_seconds)?;
    let unlock_ts = pending
        .request_ts
        .checked_add(self.cooldown_seconds)
        .ok_or(NestError::MathOverflow)?;
    if now < unlock_ts {
        return Err(NestError::CooldownActive);
    }
    self.sync_vesting(now)?;
    if pending.shares == 0 || pending.shares > self.total_shares {
        return Err(NestError::InvalidParameter);
    }
    if self.unvested_revenue > 0 {
        return Err(NestError::CooldownActive);
    }
}

```

The problem is that harvest has no signer or authority account in its account context:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-stake/src/accounts.rs#L67-L78>

```

#[derive(Accounts)]
pub struct Harvest<'info> {

```

```

#[account(mut, seeds = [b"staking"], bump = staking_state.bump)]
pub staking_state: Box<Account<'info, StakingState>>,
#[account(mut, address = staking_state.nusd_mint)]
pub nusd_mint: InterfaceAccount<'info, Mint>,
#[account(mut, address = staking_state.revenue_nusd_vault, token::mint =
↳ nusd_mint, token::authority = staking_state, token::token_program =
↳ nusd_token_program)]
pub revenue_nusd_account: InterfaceAccount<'info, TokenAccount>,
#[account(mut, address = staking_state.staking_nusd_vault, token::mint =
↳ nusd_mint, token::authority = staking_state, token::token_program =
↳ nusd_token_program)]
pub staking_nusd_vault: InterfaceAccount<'info, TokenAccount>,
#[account(address = SPL_TOKEN_PROGRAM_ID)]
pub nusd_token_program: Interface<'info, TokenInterface>,
}

```

The instruction only requires a positive amount, existing shares, and enough excess nUSD in the revenue vault:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-stake/src/instructions.rs#L146-L158>

```

pub fn harvest(ctx: Context<Harvest>, amount: u64) -> Result<()> {
    require!(amount > 0, StakeError::InvalidParameter);
    require!(
        ctx.accounts.staking_state.total_shares > 0,
        StakeError::InvalidParameter
    );
    let amount_u128 = amount as u128;
    let revenue_balance = ctx.accounts.revenue_nusd_account.amount as u128;
    let harvestable = revenue_balance
        .checked_sub(ctx.accounts.staking_state.revenue_baseline_nusd)
        .ok_or(error!(StakeError::InsufficientAssets))?;
    require!(amount_u128 <= harvestable, StakeError::InsufficientAssets);
}

```

After passing those checks, the domain harvest starts a new vesting period for the harvested amount:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/s/crates/nest-domain/src/staking.rs#L97-L116>

```

pub fn harvest(&mut self, amount: u128, now: i64) -> Result<()> {
    require_nonnegative_ts(now)?;
    if amount == 0 || self.total_shares == 0 {
        return Err(NestError::InvalidParameter);
    }
    if self.unvested_revenue > 0 && now < self.vesting_end_ts {
        return Err(NestError::InvalidParameter);
    }
    self.sync_vesting(now)?;
}

```

```

    if self.unvested_revenue != 0 {
        return Err(NestError::InvalidParameter);
    }
    self.staking_vault_nusd = checked_add(self.staking_vault_nusd, amount)?;
    self.unvested_revenue = checked_add(self.unvested_revenue, amount)?;
    self.vesting_start_ts = now;
    self.last_vesting_sync_ts = now;
    self.vesting_end_ts = now
        .checked_add(self.revenue_vesting_seconds)
        .ok_or(NestError::MathOverflow)?;
    Ok(())
}

```

Both the unstake cooldown and the revenue vesting period default to 7 days:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/crates/nest-domain/src/constants.rs#L15-L16>

```

pub const DEFAULT_COOLDOWN_SECONDS: i64 = 7 * 24 * 60 * 60;
pub const DEFAULT_REVENUE_VESTING_SECONDS: i64 = 7 * 24 * 60 * 60;

```

An attacker can exploit this as follows:

1. A victim calls `request_unstake`; their `snUSD` is burned and the pending withdrawal is created.
2. Near the cooldown boundary, the attacker transfers 1 base unit of `nUSD` directly into `revenue_nusd_vault`.
3. The attacker calls `harvest(1)`.
4. `harvest(1)` sets `unvested_revenue = 1` and `vesting_end_ts = now + 7 days`.
5. The victim's `complete_unstake` call reverts with `CooldownActive`.
6. The attacker repeats the dust harvest after each vesting period, indefinitely postponing completion at negligible recurring cost.

Ordinary users cannot clear `unvested_revenue` early. There is no public `sync_vesting` instruction, and `complete_unstake` only clears it after the full vesting period has elapsed. The only other path that can reduce `unvested_revenue` is `realize_loss`, but it requires the staking authority and burns staking vault assets, so it is not a user-side recovery path.

Impact

Pending unstakers can be denied withdrawals indefinitely. This is especially severe because their `snUSD` has already been burned during `request_unstake`, so they cannot transfer the position, cancel the pending withdrawal, or create another pending withdrawal for the same staking state. An attacker only needs to spend 1 base unit of `nUSD` per cycle, plus transaction fees, to keep affected funds locked.

Code Snippet

The existing domain regression shows that final pending unstake completion is blocked while harvested revenue remains unvested:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/crates/nest-domain/src/tests.rs#L1283-L1298>

```
#[test]
fn final_pending_unstake_waits_for_unvested_revenue() {
    let mut pool = StakingPool::default();
    let minted = pool.stake(10_000_000, 0).unwrap();
    let pending = pool.request_unstake(minted, 1).unwrap();

    let late_harvest_ts = 1 + DEFAULT_COOLDOWN_SECONDS - 10;
    pool.harvest(10_000_000, late_harvest_ts).unwrap();

    assert_eq!(
        pool.complete_unstake(pending, 1 + DEFAULT_COOLDOWN_SECONDS),
        Err(NestError::CooldownActive)
    );

    let assets = pool
        .complete_unstake(pending, late_harvest_ts +
            ↪ DEFAULT_REVENUE_VESTING_SECONDS)
        .unwrap();
}
```

The same behavior can be triggered with a dust harvest amount such as 1 once the attacker has transferred that dust into the revenue vault.

Tool Used

Manual Review

```
cargo test -p nest-domain final_pending_unstake_waits_for_unvested_revenue -- --nocapture
```

Recommendation

Do not make pending unstake completion depend on the global `unvested_revenue == 0` condition. A safer design is to snapshot the user's redeemable assets at `request_unstake` time or otherwise exclude pending withdrawals from future revenue-vesting state, so later harvests cannot block completion.

If the current completion gating is intentionally preserved, restrict harvest with an authority, minimum meaningful amount, or rate limit. However, access control alone is

weaker than removing the global vesting dependency from withdrawal completion, because users should not lose liveness after their shares have already been burned.

Discussion

ducksquadd

<https://github.com/NestUSD/contracts/commit/ba4212595e23144b28e8964bcdeb1fde7340d2>

0502lian

Fixed

Issue H-4: Mature pending exits can bypass later staking loss realization [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/issues/56>

Summary

`request_unstake` burns the user's `snUSD` and records only the pending share amount and request timestamp. Those pending shares remain in `total_shares` until `complete_unstake`, and there is no expiry after the cooldown elapses. This lets a user who has already requested an exit keep economic exposure while retaining full discretion over when to exit: on the reward side, they can wait until later revenue fully vests; on the loss side, if a loss is known but not yet realized, they can complete immediately before the authority's `realize_loss` transaction. Because `complete_unstake` prices the pending withdrawal at the current `accounted_assets / total_shares` exchange rate and is not blocked by the staking pause flag, the pending user can withdraw at the pre-loss rate and shift their fair share of the loss onto the remaining active stakers.

Vulnerability Detail

When a user requests an unstake, the domain logic only returns a `PendingWithdrawal` containing shares and `request_ts`:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/crates/nest-domain/src/staking.rs#L119-L129>

```
pub fn request_unstake(&mut self, shares: u128, now: i64) ->
↳ Result<PendingWithdrawal> {
    require_nonnegative_ts(now)?;
    self.sync_vesting(now)?;
    if shares == 0 || shares > self.total_shares {
        return Err(NestError::InvalidParameter);
    }
    Ok(PendingWithdrawal {
        shares,
        request_ts: now,
    })
}
```

The request does not snapshot a fixed asset claim and does not remove the pending shares from `total_shares`. The on-chain instruction then burns the user's `snUSD` and subtracts the shares from `total_user_shares`, but the pool-level `total_shares` used for exchange-rate accounting is unchanged:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-stake/src/instructions.rs#L195-L234>


```

let pending = pool
    .request_unstake(shares_u128, now)
    .map_err(map_stake_error)?;
apply_pool(&mut ctx.accounts.staking_state, pool);
ctx.accounts.staking_state.total_user_shares = ctx
    .accounts
    .staking_state
    .total_user_shares
    .checked_sub(pending.shares)
    .ok_or(error!(StakeError::MathOverflow))?;

let account = &mut ctx.accounts.pending_withdrawal;
account.owner = ctx.accounts.owner.key();
account.staking_state = ctx.accounts.staking_state.key();
account.shares = pending.shares;
account.request_ts = pending.request_ts;
account.completed = false;
account.bump = ctx.bumps.pending_withdrawal;

token_burn_checked(
    ctx.accounts.snusd_token_program.to_account_info(),
    ctx.accounts.snusd_mint.to_account_info(),
    ctx.accounts.owner_snusd_account.to_account_info(),
    ctx.accounts.owner.to_account_info(),
    shares,
    ctx.accounts.snusd_mint.decimals,
    &[],
)?;

```

Once the cooldown has elapsed, `complete_unstake` has no maximum completion timestamp or expiry check. It only verifies `now >= request_ts + cooldown_seconds`, then prices the pending shares using the current pool exchange rate:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/crates/nest-domain/src/staking.rs#L131-L154>

```

pub fn complete_unstake(&mut self, pending: PendingWithdrawal, now: i64) ->
    Result<u128> {
    require_nonnegative_ts(now)?;
    require_nonnegative_ts(pending.request_ts)?;
    require_nonnegative_ts(self.cooldown_seconds)?;
    let unlock_ts = pending
        .request_ts
        .checked_add(self.cooldown_seconds)
        .ok_or(NestError::MathOverflow)?;
    if now < unlock_ts {
        return Err(NestError::CooldownActive);
    }
    self.sync_vesting(now)?;
}

```

```

    if pending.shares == 0 || pending.shares > self.total_shares {
        return Err(NestError::InvalidParameter);
    }
    if self.unvested_revenue > 0 {
        return Err(NestError::CooldownActive);
    }
    let accounted = self.accounted_assets()?;
    let assets = mul_div_down(pending.shares, accounted, self.total_shares)?;
    self.total_shares = checked_sub(self.total_shares, pending.shares)?;
    self.staking_vault_nusd = checked_sub(self.staking_vault_nusd, assets)?;
    Ok(assets)
}

```

The public on-chain `complete_unstake` wrapper also does not check `staking_state.paused`, unlike `stake` and `request_unstake`:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-stake/src/instructions.rs#L245-L293>

```

pub fn complete_unstake(ctx: Context<CompleteUnstake>) -> Result<()> {
    require!(
        !ctx.accounts.pending_withdrawal.completed,
        StakeError::AlreadyCompleted
    );
    let mut pool = domain_pool(&ctx.accounts.staking_state);
    let now = Clock::get()?.unix_timestamp;
    let assets = pool
        .complete_unstake(
            domain::PendingWithdrawal {
                shares: ctx.accounts.pending_withdrawal.shares,
                request_ts: ctx.accounts.pending_withdrawal.request_ts,
            },
            now,
        )
        .map_err(map_stake_error)?;
    apply_pool(&mut ctx.accounts.staking_state, pool);
    ctx.accounts.pending_withdrawal.assets_redeemed = assets;
    ctx.accounts.pending_withdrawal.completed = true;
}

```

Loss realization is authority-only and can occur later than a public completion transaction:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-stake/src/instructions.rs#L298-L307>

```

pub fn realize_loss(ctx: Context<RealizeLoss>, amount: u64) -> Result<()> {
    require!(amount > 0, StakeError::InvalidParameter);
    assert_authority(&ctx.accounts.staking_state, &ctx.accounts.authority)?;
    let mut pool = domain_pool(&ctx.accounts.staking_state);
    pool.realize_loss(amount as u128).map_err(map_stake_error)?;
}

```

```

apply_pool(&mut ctx.accounts.staking_state, pool);
ctx.accounts.staking_state.realized_loss_nusd = ctx
    .accounts
    .staking_state
    .realized_loss_nusd

```

This gives matured pending exits a timing option. The issue is not just that withdrawals are delayed by a cooldown; it is that a user who has already requested an exit keeps economic exposure while the exit time remains entirely user-controlled. If later revenue is harvested, the pending user can wait until that revenue fully vests and then redeem at the higher exchange rate. If a later loss is known before the authority transaction is executed, the same user can instead complete first and redeem at the pre-loss exchange rate. The remaining stakers then absorb the later `realize_loss` despite the pending shares having remained in the denominator and having been economically exposed up to that point.

Example:

1. Alice stakes 100 nUSD and Bob stakes 100 nUSD, leaving 200 assets and 200 shares.
2. Bob requests to unstake 100 shares and waits past the cooldown, but does not complete.
3. A 100 nUSD staking loss becomes known.
4. If `realize_loss(100)` executes first, Bob receives $100 * 100 / 200 = 50$, and Alice's remaining position is worth 50.
5. If Bob completes first, he receives $100 * 200 / 200 = 100$; after `realize_loss(100)`, Alice is left with 0.

Existing tests already confirm that pending shares remain economic shares while pending. A loss before completion is socialized to pending shares:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/crates/nest-domain/src/tests.rs#L1198-L1210>

```

#[test]
fn staking_losses_socialize_to_pending_shares() {
    let mut pool = StakingPool::default();
    pool.stake(100_000_000, 0).unwrap();
    let pending = pool.request_unstake(50_000_000, 1).unwrap();
    assert_eq!(pool.total_shares, 100_000_000);
    pool.realize_loss(20_000_000).unwrap();
    let assets = pool
        .complete_unstake(pending, 1 + DEFAULT_COOLDOWN_SECONDS)
        .unwrap();
    assert_eq!(assets, 40_000_000);
}

```

Pending shares also remain in the denominator for later vested revenue:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/crates/nest-domain/src/tests.rs#L1213-L1227>

```
#[test]
fn staking_pending_shares_remain_in_revenue_denominator() {
    let mut pool = StakingPool::default();
    pool.stake(100_000_000, 0).unwrap();
    let pending = pool.request_unstake(50_000_000, 1).unwrap();
    assert_eq!(pool.total_shares, 100_000_000);

    pool.harvest(10_000_000, 2).unwrap();
    let assets = pool
        .complete_unstake(pending, 2 + DEFAULT_REVENUE_VESTING_SECONDS)
        .unwrap();

    assert_eq!(assets, 55_000_000);
}
```

Since these pending shares remain economically active and can benefit from later vested revenue, they should also be unable to selectively avoid later known losses by choosing the completion ordering.

Impact

Active stakers can absorb losses that should have been socialized across both active and pending shares. A large matured pending exit can withdraw at the pre-loss exchange rate, leaving remaining active stakers to take most or all of the realized loss. The attack is especially damaging when loss information is observable before the authority's `realize_loss` transaction is finalized, because completion is public, unexpired after cooldown, and not blocked by the staking pause flag.

Code Snippet

The following domain-level regression demonstrates the ordering dependency:

```
#[test]
fn matured_pending_unstake_can_bypass_later_realize_loss() {
    let mut fair = StakingPool::default();
    fair.stake(100_000_000, 0).unwrap();
    let bob_shares = fair.stake(100_000_000, 0).unwrap();
    let pending = fair.request_unstake(bob_shares, 1).unwrap();

    fair.realize_loss(100_000_000).unwrap();
    let bob_assets = fair
        .complete_unstake(pending, 1 + DEFAULT_COOLDOWN_SECONDS)
        .unwrap();

    assert_eq!(bob_assets, 50_000_000);
}
```

```

    assert_eq!(fair.staking_vault_nusd, 50_000_000);

    let mut exploited = StakingPool::default();
    exploited.stake(100_000_000, 0).unwrap();
    let bob_shares = exploited.stake(100_000_000, 0).unwrap();
    let pending = exploited.request_unstake(bob_shares, 1).unwrap();

    let bob_assets = exploited
        .complete_unstake(pending, 1 + DEFAULT_COOLDOWN_SECONDS)
        .unwrap();
    exploited.realize_loss(100_000_000).unwrap();

    assert_eq!(bob_assets, 100_000_000);
    assert_eq!(exploited.staking_vault_nusd, 0);
}

```

Tool Used

Manual Review

Recommendation

Remove the timing option from pending withdrawals. Viable approaches include:

1. Snapshot and reserve the redeemable asset claim at `request_unstake` time, then remove those shares from the live exchange-rate denominator.
2. Keep pending withdrawals share-denominated, but make `realize_loss` explicitly apply to pending withdrawals so matured exits cannot avoid their proportional loss.
3. Add an expiry or force-completion mechanism for matured pending exits, and consider making `complete_unstake` respect the staking pause flag so the authority can halt completions while a known loss is being realized.

The key invariant is that a withdrawal should not remain economically exposed to future rewards while retaining the ability to bypass future losses by selecting the completion order.

Discussion

ducksquadd

<https://github.com/NestUSD/contracts/commit/ba4212595e23144b28e8964bcdeb1fde7340d2>

0502lian

fixed

Issue H-5: Staker target revenue accrues using current stake for the whole stale interval [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/issues/58>

Summary

The core revenue router caps staker allocations by accruing `staker_target_revenue_due`, but that accrual is synchronized only when revenue is routed. The sync uses the current `staking_assets` balance for the whole elapsed interval since `staker_target_last_accrual_ts`. Staking and unstaking change the staking vault balance without first synchronizing this core-side timer.

A user can stake immediately before a repayment or yield realization and make the protocol treat that fresh stake as if it existed throughout the entire quiet period. This inflates the staker target revenue due and routes realized fee or yield to the staker revenue vault instead of protocol revenue.

Vulnerability Detail

When realized revenue is routed, the target APR path calls `sync_staker_target_revenue` before deciding how much revenue stakers may receive:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/helpers/revenue.rs#L75-L108>

```
fn route_targeted_staker_revenue(
    protocol: &mut Protocol,
    staking_assets: u128,
    now: i64,
    raw_staker_revenue: u128,
) -> Result<(u128, u128)> {
    if protocol.staker_target_apr_bps == 0 {
        protocol.realized_revenue_for_stakers = protocol
            .realized_revenue_for_stakers
            .checked_add(raw_staker_revenue)
            .ok_or(error!(CoreError::MathOverflow))?;
        return Ok((raw_staker_revenue, 0));
    }

    sync_staker_target_revenue(protocol, staking_assets, now)?;
    let staker_delta = core::cmp::min(raw_staker_revenue,
        ↪ protocol.staker_target_revenue_due);
```

The sync calculates the newly accrued target revenue from the current `staking_assets` and the entire elapsed time since the last accrual timestamp:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/helpers/revenue.rs#L111-L145>

```
fn sync_staker_target_revenue(
    protocol: &mut Protocol,
    staking_assets: u128,
    now: i64,
) -> Result<()> {
    ...
    let elapsed = (now - protocol.staker_target_last_accrual_ts) as u128;
    protocol.staker_target_last_accrual_ts = now;
    if staking_assets == 0 || protocol.staker_target_apr_bps == 0 {
        return Ok(());
    }
    let accrued = staking_assets
        .checked_mul(protocol.staker_target_apr_bps as u128)
        .and_then(|value| value.checked_mul(elapsed))
        .ok_or(error!(CoreError::MathOverflow))?
        .checked_div(domain::BPS_DENOMINATOR)
        .ok_or(error!(CoreError::MathOverflow))?
        .checked_div(domain::SECONDS_PER_YEAR)
        .ok_or(error!(CoreError::MathOverflow))?;
    protocol.staker_target_revenue_due = protocol
        .staker_target_revenue_due
        .checked_add(accrued)
        .ok_or(error!(CoreError::MathOverflow))?;
```

That `staking_assets` value is read at repayment time from the current staking vault state:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/ix/cdp.rs#L210-L260>

```
let staking_assets = staking_vault_nusd_from_account(
    &ctx.accounts.staking_state.to_account_info(),
    ctx.accounts.protocol.nusd_mint,
)?;
...
let (staker_delta, protocol_delta) = route_targeted_staker_revenue(
    &mut ctx.accounts.protocol,
    staking_assets,
    now,
    staker_delta,
)?;
```

However, staking itself only updates the staking pool and capacity-checks against the core protocol account. It does not synchronize the core target accrual timestamp using the pre-stake balance:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/ix/cdp.rs#L210-L260>

[s/programs/nest-stake/src/instructions.rs#L62-L89](https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/crates/nest-domain/src/instructions.rs#L62-L89)

```
pub fn stake(ctx: Context<Stake>, amount: u64) -> Result<> {
    require!(amount > 0, StakeError::InvalidParameter);
    require!(!ctx.accounts.staking_state.paused, StakeError::Paused);
    let was_empty = ctx.accounts.staking_state.total_shares == 0;
    let mut pool = domain_pool(&ctx.accounts.staking_state);
    let now = Clock::get()?.unix_timestamp;
    let minted_shares = pool.stake(amount as u128, now).map_err(map_stake_error)?;
    if let Some(capacity) = staking_capacity_from_protocol(
        &ctx.accounts.protocol.to_account_info(),
        ctx.accounts.staking_state.nusd_mint,
    )? {
        require!(
            pool.staking_vault_nusd <= capacity,
            StakeError::StakeCapacityExceeded
        );
    }
    apply_pool(&mut ctx.accounts.staking_state, pool);
}
```

CDP fees can accrue for a long period without being routed to stakers or protocol revenue. Fee accrual only increases `vault.accrued_fee` and `protocol.total_uncollected_fees`:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/crates/nest-domain/src/cdp.rs#L175-L190>

```
vault.accrued_fee = checked_add(vault.accrued_fee, fee)?;
protocol.total_debt = checked_add(protocol.total_debt, fee)?;
protocol.total_uncollected_fees = checked_add(protocol.total_uncollected_fees,
    ↪ fee)?;
vault.last_accrual_ts = now;
```

Those fees are routed only when they are actually paid:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/crates/nest-domain/src/cdp.rs#L288-L300>

```
let fee_paid = core::cmp::min(amount, vault.accrued_fee);
vault.accrued_fee = checked_sub(vault.accrued_fee, fee_paid)?;
protocol.total_uncollected_fees = checked_sub(protocol.total_uncollected_fees,
    ↪ fee_paid)?;
protocol.total_debt = checked_sub(protocol.total_debt, fee_paid)?;
protocol.route_realized_fee(fee_paid)?;
```

As a result, after `staker_target_last_accrual_ts` has been initialized, an attacker can wait for a long period with little or no revenue routing, stake a large amount immediately before a repayment or PSM yield realization, and cause the next sync to accrue:


```
current_staking_assets * staker_target_apr_bps * full_stale_elapsed_time
```

instead of accruing over the historical staking balance that was actually present during that period.

For example, if the target APR is 10%, the target timer is 30 days stale, and an attacker stakes 1,000,000 nUSD just before a repayment, the protocol can add roughly:

```
1,000,000 * 10% * 30 / 365 = 8,219 nUSD
```

to `staker_target_revenue_due`, even if that stake was present only for a few seconds. The next realized fees or yield can then be routed to the staker revenue vault up to this inflated due amount.

The first ever sync only initializes `staker_target_last_accrual_ts` and returns, so the issue applies once the target accrual timestamp has already been initialized and later becomes stale.

Impact

Protocol revenue can be diverted to the staker revenue vault. A dominant or sole just-in-time staker can capture a large share of realized fees or yield that should have gone to protocol revenue, reducing protocol or buyback revenue and diluting historical stakers who were actually present during the accrual period.

Code Snippet

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/helpers/revenue.rs#L128-L144>

```
let elapsed = (now - protocol.staker_target_last_accrual_ts) as u128;
protocol.staker_target_last_accrual_ts = now;
if staking_assets == 0 || protocol.staker_target_apr_bps == 0 {
    return Ok(());
}
let accrued = staking_assets
    .checked_mul(protocol.staker_target_apr_bps as u128)
    .and_then(|value| value.checked_mul(elapsed))
    .ok_or(error!(CoreError::MathOverflow))?
    .checked_div(domain::BPS_DENOMINATOR)
    .ok_or(error!(CoreError::MathOverflow))?
    .checked_div(domain::SECONDS_PER_YEAR)
    .ok_or(error!(CoreError::MathOverflow))?;
protocol.staker_target_revenue_due = protocol
    .staker_target_revenue_due
```

```
.checked_add(accrued)
.ok_or(error!(CoreError::MathOverflow))?;
```

Tool Used

Manual Review

Recommendation

Synchronize the core staker target accrual whenever staking assets change, using the staking balance that existed before the change. At minimum, stake and unstake flows should settle `staker_target_revenue_due` up to the current timestamp before mutating `staking_vault_nusd`.

A more robust design is to move target APR accounting into a staking-aware accumulator or checkpoint system so the accrued target is based on time-weighted historical staking assets rather than the staking balance observed at revenue realization time.

Discussion

ducksquadd

<https://github.com/NestUSD/contracts/commit/e6c725300046531375727124761c627e00e262c2>

0502lian

Fixed

Issue H-6: First staker can inflate snUSD exchange rate via donated harvestable revenue and capture later depositor rounding loss [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/issues/62>

Summary

A first staker can stake a dust amount, transfer nUSD directly into `revenue_nusd_vault`, and call permissionless `harvest()` to make that attacker-funded balance part of the staking pool's recorded assets. New deposits are then priced against the inflated `stake_entry_assets` value.

Because `stake()` mints shares with floor division and the external `stake(ctx, amount)` instruction has no caller-supplied `min_shares` or slippage protection, a later depositor can receive materially fewer snUSD shares than their deposit should reasonably buy. After the harvested revenue vests and the cooldown elapses, the first staker can redeem their dust share for part of the later depositor's principal.

Vulnerability Detail

When the first user stakes into an empty pool, shares are minted 1:1 with the deposited amount:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/crates/nest-domain/src/staking.rs#L75-L88>

```
pub fn stake(&mut self, amount: u128, now: i64) -> Result<u128> {
    if amount == 0 {
        return Err(NestError::InvalidParameter);
    }
    self.sync_vesting(now)?;
    let entry_assets = self.stake_entry_assets()?;
    let shares = if self.total_shares == 0 {
        amount
    } else {
        if entry_assets == 0 {
            return Err(NestError::InvalidParameter);
        }
        mul_div_down(amount, self.total_shares, entry_assets)?
    };
};
```

For a non-empty pool, the same function uses floor division. It only rejects `shares == 0`, so any positive rounded-down share amount is accepted:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/crates/nest-domain/src/staking.rs#L87-L94>

```
        mul_div_down(amount, self.total_shares, entry_assets)?
    };
    if shares == 0 {
        return Err(NestError::InvalidParameter);
    }
    self.staking_vault_nusd = checked_add(self.staking_vault_nusd, amount)?;
    self.total_shares = checked_add(self.total_shares, shares)?;
    Ok(shares)
```

The entry price denominator is `stake_entry_assets()`:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/crates/nest-domain/src/staking.rs#L47-L49>

```
pub fn stake_entry_assets(&self) -> Result<u128> {
    checked_sub(self.staking_vault_nusd, self.reserved_pending_claims)
}
```

This value includes harvested revenue even while it is still unvested, because `harvest()` adds the amount to `staking_vault_nusd` and `unvested_revenue`:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/crates/nest-domain/src/staking.rs#L97-L116>

```
pub fn harvest(&mut self, amount: u128, now: i64) -> Result<()> {
    require_nonnegative_ts(now)?;
    if amount == 0 || self.total_shares == 0 {
        return Err(NestError::InvalidParameter);
    }
    if self.unvested_revenue > 0 && now < self.vesting_end_ts {
        return Err(NestError::InvalidParameter);
    }
    self.sync_vesting(now)?;
    if self.unvested_revenue != 0 {
        return Err(NestError::InvalidParameter);
    }
    self.staking_vault_nusd = checked_add(self.staking_vault_nusd, amount)?;
    self.unvested_revenue = checked_add(self.unvested_revenue, amount)?;
    self.vesting_start_ts = now;
    self.last_vesting_sync_ts = now;
    self.vesting_end_ts = now
        .checked_add(self.revenue_vesting_seconds)
        .ok_or(NestError::MathOverflow)?;
    Ok(())
}
```

The on-chain `harvest()` instruction accepts any amount up to the token balance sitting

above revenue_baseline_nusd:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-stake/src/instructions.rs#L146-L160>

```
pub fn harvest(ctx: Context<Harvest>, amount: u64) -> Result<()> {
    require!(amount > 0, StakeError::InvalidParameter);
    require!(
        ctx.accounts.staking_state.total_shares > 0,
        StakeError::InvalidParameter
    );
    let amount_u128 = amount as u128;
    let revenue_balance = ctx.accounts.revenue_nusd_account.amount as u128;
    let harvestable = revenue_balance
        .checked_sub(ctx.accounts.staking_state.revenue_baseline_nusd)
        .ok_or(error!(StakeError::InsufficientAssets))?;
    require!(amount_u128 <= harvestable, StakeError::InsufficientAssets);
    let mut pool = domain_pool(&ctx.accounts.staking_state);
    let now = Clock::get()?.unix_timestamp;
    pool.harvest(amount_u128, now).map_err(map_stake_error)?;
```

There is no check that the harvestable amount corresponds to a core-program staker revenue liability. Any user can transfer SPL tokens directly into the revenue_nusd_vault, and the instruction only observes the resulting balance.

The Harvest account context has no signer:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-stake/src/accounts.rs#L67-L78>

```
#[derive(Accounts)]
pub struct Harvest<'info> {
    #[account(mut, seeds = [b"staking"], bump = staking_state.bump)]
    pub staking_state: Box<Account<'info, StakingState>>,
    #[account(mut, address = staking_state.nusd_mint)]
    pub nusd_mint: InterfaceAccount<'info, Mint>,
    #[account(mut, address = staking_state.revenue_nusd_vault, token::mint =
        ↪ nusd_mint, token::authority = staking_state, token::token_program =
        ↪ nusd_token_program)]
    pub revenue_nusd_account: InterfaceAccount<'info, TokenAccount>,
    #[account(mut, address = staking_state.staking_nusd_vault, token::mint =
        ↪ nusd_mint, token::authority = staking_state, token::token_program =
        ↪ nusd_token_program)]
    pub staking_nusd_vault: InterfaceAccount<'info, TokenAccount>,
    #[account(address = SPL_TOKEN_PROGRAM_ID)]
    pub nusd_token_program: Interface<'info, TokenInterface>,
}
```

The external staking instruction also has no min_shares argument, so the depositor cannot reject unexpectedly bad pricing:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-stake/src/instructions.rs#L62-L68>

```
pub fn stake(ctx: Context<Stake>, amount: u64) -> Result<> {
    require!(amount > 0, StakeError::InvalidParameter);
    require!(!ctx.accounts.staking_state.paused, StakeError::Paused);
    let was_empty = ctx.accounts.staking_state.total_shares == 0;
    let mut pool = domain_pool(&ctx.accounts.staking_state);
    let now = Clock::get()?.unix_timestamp;
    let minted_shares = pool.stake(amount as u128, now).map_err(map_stake_error)?;
```

Attack scenario:

1. The pool is empty.
2. The attacker stakes 1 atomic nUSD unit and receives 1 share.
3. The attacker transfers 50_000_000 atomic nUSD units directly into revenue_nusd_vault.
4. The attacker calls `harvest(50_000_000)`.
5. The staking pool now has 50_000_001 recorded entry assets and only 1 share.
6. A victim stakes 100_000_000 atomic units.
7. Victim shares are $\text{floor}(100_000_000 * 1 / 50_000_001) = 1$.
8. After vesting, total assets are 150_000_001 and total shares are 2.
9. The attacker redeems 1 share for 75_000_000 atomic units.

The attacker spent 50_000_001 atomic units (1 first stake plus 50_000_000 donated harvestable revenue) and receives 75_000_000, profiting 24_999_999 atomic units from the victim's deposit.

Impact

Victim stakers can lose a concrete portion of deposited nUSD to the first staker. With the values above, a victim depositing 100_000_000 atomic units can redeem only 75_000_001, losing 24_999_999 atomic units. The attacker's matching profit is 24_999_999 atomic units with 50_000_001 atomic units of temporary capital.

This is a direct loss of user principal caused by manipulated share pricing and rounding. The issue is most acute for an empty or near-empty pool where the first staker controls a very small total share supply.

Code Snippet

The following PoC was run against the checked-out repository by linking directly to the project's `nest-domain` crate, matching the style of the repository's existing domain tests.

PoC file:

```
use nest_domain::{StakingPool, DEFAULT_COOLDOWN_SECONDS,
↳ DEFAULT_REVENUE_VESTING_SECONDS};

fn main() {
    let attacker_seed_stake = 1_u128;
    let attacker_donated_revenue = 50_000_000_u128;
    let victim_deposit = 100_000_000_u128;

    let mut pool = StakingPool::default();

    let attacker_shares = pool.stake(attacker_seed_stake, 0).unwrap();
    assert_eq!(attacker_shares, 1);
    assert_eq!(pool.total_shares, 1);
    assert_eq!(pool.staking_vault_nusd, 1);

    // On-chain reachability model: the first stake snapshots the empty revenue
    // vault as the baseline. A later direct SPL transfer to the revenue vault
    // makes exactly the donated amount harvestable.
    let revenue_baseline_nusd = 0_u128;
    let revenue_nusd_vault_amount = revenue_baseline_nusd +
↳ attacker_donated_revenue;
    let harvestable = revenue_nusd_vault_amount
        .checked_sub(revenue_baseline_nusd)
        .unwrap();
    assert_eq!(harvestable, attacker_donated_revenue);

    pool.harvest(harvestable, 1).unwrap();
    assert_eq!(pool.staking_vault_nusd, 50_000_001);
    assert_eq!(pool.stake_entry_assets().unwrap(), 50_000_001);
    assert_eq!(pool.unvested_revenue, attacker_donated_revenue);

    let victim_shares = pool.stake(victim_deposit, 2).unwrap();
    assert_eq!(victim_shares, 1);
    assert_eq!(pool.total_shares, 2);
    assert_eq!(pool.staking_vault_nusd, 150_000_001);

    let vesting_done = 1 + DEFAULT_REVENUE_VESTING_SECONDS;
    pool.sync_vesting(vesting_done).unwrap();
    assert_eq!(pool.unvested_revenue, 0);
    assert_eq!(pool.accounted_assets().unwrap(), 150_000_001);

    let attacker_pending = pool.request_unstake(attacker_shares,
↳ vesting_done).unwrap();
    let attacker_redeemed = pool
        .complete_unstake(attacker_pending, vesting_done + DEFAULT_COOLDOWN_SECONDS)
        .unwrap();
    assert_eq!(attacker_redeemed, 75_000_000);
```

```

let attacker_cost = attacker_seed_stake + attacker_donated_revenue;
let attacker_profit = attacker_redeemed.checked_sub(attacker_cost).unwrap();
assert_eq!(attacker_profit, 24_999_999);

let victim_pending = pool
    .request_unstake(victim_shares, vesting_done + DEFAULT_COOLDOWN_SECONDS)
    .unwrap();
let victim_redeemed = pool
    .complete_unstake(
        victim_pending,
        vesting_done + DEFAULT_COOLDOWN_SECONDS * 2,
    )
    .unwrap();
let victim_loss = victim_deposit.checked_sub(victim_redeemed).unwrap();
assert_eq!(victim_redeemed, 75_000_001);
assert_eq!(victim_loss, attacker_profit);

println!(
    "valid=true attacker_shares={} victim_shares={} attacker_redeemed={}
    ↪ attacker_cost={} attacker_profit={} victim_redeemed={} victim_loss={}",
    attacker_shares,
    victim_shares,
    attacker_redeemed,
    attacker_cost,
    attacker_profit,
    victim_redeemed,
    victim_loss
);
}

```

Run steps:

```

cargo run --manifest-path security-scan-artifacts/2026-07-nestusd/manual-validation
↪ -first-staker-rounding-20260709/artifacts/05_findings/CAND-FIRST-STAKER-HARVEST-
↪ ROUNDING-001/validation_artifacts/Cargo.toml

```

Observed result:

```

valid=true attacker_shares=1 victim_shares=1 attacker_redeemed=75000000
↪ attacker_cost=50000001 attacker_profit=24999999 victim_redeemed=75000001
↪ victim_loss=24999999

```

Additional repository test command:

```

cargo test -p nest-domain --manifest-path contracts/Cargo.toml

```

Observed result:


```
45 passed; 0 failed.
```

Tool Used

Manual Review

Focused Rust PoC:

```
cargo run --manifest-path security-scan-artifacts/2026-07-nestusd/manual-validation
↳ -first-staker-rounding-20260709/artifacts/05_findings/CAND-FIRST-STAKER-HARVEST-
↳ ROUNDING-001/validation_artifacts/Cargo.toml
```

Repository test suite:

```
cargo test -p nest-domain --manifest-path contracts/Cargo.toml
```

Recommendation

Do not allow arbitrary token balances in `revenue_nusd_vault` to become staker revenue only because they exceed `revenue_baseline_nusd.harvest()` should be bounded by core-program accounting for realized staker revenue, not by raw token-account balance.

Also add caller-side slippage protection to staking, for example `stake(ctx, amount, min_shares_out)`, and revert if the minted `snUSD` shares are below the caller's minimum.

To reduce first-depositor exchange-rate manipulation, consider requiring a meaningful minimum initial stake, minting permanently locked dead shares on initialization, or otherwise preventing a dust first staker from controlling the initial share price.

Discussion

ducksquadd

<https://github.com/NestUSD/contracts/commit/85aa0106a2e0a9f90996cf3bf71457155884070f>

0502lian

Fixed

Issue H-7: Two-step liquidation confiscates borrower surplus [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/issues/63>

Summary

The two-step liquidation path seizes the borrower's entire collateral balance whenever the vault qualifies for full liquidation. Even if the vault is still overcollateralized in absolute terms, the borrower receives no refund. At settlement, all value above principal debt is routed as protocol/staker revenue instead of being returned to the borrower.

Vulnerability Detail

`start_liquidation_with_oracle()` only allows two-step liquidation when `full_liquidation == true`:

https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/ix/liquidation/two_step.rs#L30-36

```
let (_, full_liquidation) = domain::max_liquidation_repay(
    domain_vault(&ctx.accounts.vault),
    collateral_value,
    domain_params(&ctx.accounts.collateral_config, &ctx.accounts.protocol),
)
.map_err(map_core_error)?;
require!(full_liquidation, CoreError::InvalidParameter);
```

However, after this check, it records and seizes the full collateral amount:

https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/ix/liquidation/two_step.rs#L38-48

```
let principal_debt = ctx.accounts.vault.principal_debt;
let accrued_fee = ctx.accounts.vault.accrued_fee;
let collateral_raw = ctx.accounts.vault.collateral_raw;
ctx.accounts.liquidation_receipt.protocol = ctx.accounts.protocol.key();
ctx.accounts.liquidation_receipt.collateral_config =
    ↪ ctx.accounts.collateral_config.key();
ctx.accounts.liquidation_receipt.vault = ctx.accounts.vault.key();
ctx.accounts.liquidation_receipt.liquidator = ctx.accounts.liquidator.key();
ctx.accounts.liquidation_receipt.collateral_mint =
    ↪ ctx.accounts.collateral_mint.key();
ctx.accounts.liquidation_receipt.collateral_raw = collateral_raw;
ctx.accounts.liquidation_receipt.min_settlement_usdc = collateral_value;
ctx.accounts.liquidation_receipt.principal_debt = principal_debt;
```

The vault is then zeroed and the entire collateral balance is transferred to the liquidator:

https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/ix/liquidation/two_step.rs#L83-108

```
ctx.accounts.collateral_config.total_deposits_raw = ctx
    .accounts
    .collateral_config
    .total_deposits_raw
    .checked_sub(collateral_raw)
    .ok_or(error!(CoreError::MathOverflow))?;
ctx.accounts.vault.collateral_raw = 0;
ctx.accounts.vault.principal_debt = 0;
ctx.accounts.vault.accrued_fee = 0;
ctx.accounts.vault.last_accrual_ts = now;

let collateral_seized = u128_to_u64(collateral_raw)?;
let collateral_vault_before = ctx.accounts.collateral_vault.amount;
let liquidator_collateral_before =
    ↪ ctx.accounts.liquidator_collateral_account.amount;
let bump = [ctx.accounts.protocol.bump];
let signer_seeds: &[&[u8]] = &[&[b"protocol", &bump]];
token_transfer_checked(
    ctx.accounts.collateral_token_program.to_account_info(),
    ctx.accounts.collateral_vault.to_account_info(),
    ctx.accounts.collateral_mint.to_account_info(),
    ctx.accounts.liquidator_collateral_account.to_account_info(),
    ctx.accounts.protocol.to_account_info(),
    collateral_seized,
    ctx.accounts.collateral_mint.decimals,
    signer_seeds,
)?;
```

At settlement, the liquidator must pay at least the recorded full collateral value:

https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/ix/liquidation/two_step.rs#L134-144

```
let principal_debt = ctx.accounts.liquidation_receipt.principal_debt;
let min_settlement_usdc = ctx.accounts.liquidation_receipt.min_settlement_usdc;
let settlement_floor = core::cmp::max(principal_debt, min_settlement_usdc);

require!(
    usdc_amount as u128 >= settlement_floor,
    CoreError::InsufficientLiquidationProceeds
);
```

Any settlement amount above principal is then treated as revenue:

https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/ix/liquidation/two_step.rs#L134-144

[s/programs/nest-core/src/ix/liquidation/two_step.rs#L206-221](#)

```
let revenue_amount = (usdc_amount as u128)
    .checked_sub(principal_debt)
    .ok_or(error!(CoreError::MathOverflow))?;
if revenue_amount > 0 {
    let staking_assets = staking_vault_nusd_from_account(
        &ctx.accounts.staking_state.to_account_info(),
        ctx.accounts.protocol.nusd_mint,
    );
    let (insurance_delta_u64, staker_delta_u64, protocol_delta_u64,
        ↪ minted_nusd_u64) =
        apply_realized_psm_yield_accounting(
            &mut ctx.accounts.protocol,
            ctx.accounts.psm_usdc_vault.amount,
            u128_to_u64(revenue_amount)?,
            staking_assets,
            Clock::get()?.unix_timestamp,
        );
}
```

This differs from the instant liquidation path, where the seized collateral is capped at approximately repaid debt + liquidation penalty, and excess collateral remains with the borrower:

```
let keeper_raw =
    collateral_raw_for_usd_value(keeper_value, token_decimals,
    ↪ raw_token_safe_price_e8)?;

let (keeper_raw, exhausts_collateral) = if vault.collateral_raw >= keeper_raw {
    let exhausts_collateral = full && vault.collateral_raw == keeper_raw;
    (keeper_raw, exhausts_collateral)
} else {
    if !full {
        return Err(NestError::InsufficientCollateral);
    }
    (vault.collateral_raw, true)
};
```

Example:

Debt:	100 nUSD
Collateral value:	140 USDC
Liquidation threshold:	55%
Full liquidation threshold:	80%
Liquidation penalty:	8%

Health factor:

```
140 * 0.55 / 100 = 0.77
```

So the vault qualifies for full liquidation. But it is still overcollateralized in absolute terms. The instant path would seize roughly:

```
100 + 8 = 108 USDC
```

and leave about 32 USDC worth of collateral for the borrower. The two-step path instead seizes the full 140 USDC, settles 100 USDC as principal, and routes the remaining 40 USDC as protocol/staker revenue.

Impact

Borrowers can lose all residual collateral in overcollateralized full liquidations executed through the two-step path.

Tool Used

Manual Review

Recommendation

Cap the collateral seized in two-step liquidation to the amount needed to cover:

```
principal debt + accrued fees + liquidation penalty
```

Discussion

ducksquaddd

Yeah this was fixed yesterday <https://github.com/NestUSD/contracts/commit/5154b6b04ee7cba199511819421707eaf176f43b>

0502lian

Fixed

Issue H-8: Oracle refresh can roll the shared snapshot back to an older price and overvalue collateral [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/issues/70>

Summary

Both permissionless oracle refresh paths validate an incoming observation against the current time, but neither compares its `publish_time` with the observation already stored in the shared `OracleSnapshot`. Consequently, an older signed observation can overwrite a newer observation while the older one remains within the configured freshness window.

During a price decline, a vault owner can replay an older high price and immediately call `mint_nusd_with_oracle` or `withdraw_with_oracle`. Those operations value collateral from the rolled-back snapshot, allowing the owner to mint unbacked nUSD or remove collateral that should secure existing debt. The same snapshot also affects both liquidation paths.

Vulnerability Detail

The Lazer refresh verifies the supplied message and validates the decoded price. It then assigns the observation directly to `oracle.xstock_usd` without requiring its timestamp to be newer than the currently stored timestamp:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/ix/oracle.rs#L20-L38>

```
let verified = pyth_lazer_solana_contract::cpi::verify_message(
    cpi_ctx,
    message_data,
    ed25519_instruction_index,
    signature_index,
)?;
let verified = verified.get();
let xstock_usd = read_lazer_price(&verified.payload,
    ↪ &ctx.accounts.collateral_config)?;
let clock = Clock::get()?;
validate_xstock_oracle_price(
    &ctx.accounts.collateral_config,
    xstock_usd,
    clock.unix_timestamp,
)?;

let oracle = &mut ctx.accounts.oracle;
oracle.protocol = ctx.accounts.protocol.key();
```

```
oracle.collateral_config = ctx.accounts.collateral_config.key();
oracle.xstock_usd = xstock_usd;
```

The signed-price path has the same state transition. Signature, feed, expiry, and current freshness are checked, but the accepted value unconditionally replaces the snapshot:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/ix/oracle.rs#L49-L95>

```
verify_ed25519_instruction(
    &ctx.accounts.instructions_sysvar,
    ed25519_instruction_index,
    &ctx.accounts.nest_price_signer.signer,
    &signed_message,
)?;

// ...validate the payload and current-time expiry...

validate_xstock_oracle_price(
    &ctx.accounts.collateral_config,
    xstock_usd,
    clock.unix_timestamp,
)?;

let oracle = &mut ctx.accounts.oracle;
oracle.protocol = ctx.accounts.protocol.key();
oracle.collateral_config = ctx.accounts.collateral_config.key();
oracle.xstock_usd = xstock_usd;
```

Neither refresh context requires an administrator or designated oracle updater. Each requires only an arbitrary payer signer, so anyone possessing a previously valid message can perform the overwrite:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/accounts/oracle.rs#L1-L44>

The price policy only rejects an observation when its age relative to the current clock exceeds `max_staleness_seconds`; it does not enforce monotonicity relative to snapshot state:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/crates/nest-domain/src/oracle.rs#L47-L68>

```
let age = if future_skew > 0 {
    0
} else {
    now.checked_sub(price.publish_time)
        .ok_or(NestError::OracleStale)?
};
if age > policy.max_staleness_seconds {
```

```
    return Err(NestError::OracleStale);  
}
```

The configured xStock freshness limit can be as high as 120 seconds. Within that interval, multiple observations can therefore all be individually fresh even though accepting them out of order rolls protocol state backward.

Pyth's official Solana/SVM consumer guide explicitly treats timestamp monotonicity as a responsibility of the consuming program, separate from signature verification. After verifying and parsing a Lazer message, the reference implementation rejects any update whose timestamp is not strictly newer than the timestamp already stored in program state:

<https://docs.pyth.network/price-feeds/pro/integrate-as-consumer/svm>

```
if data.timestamp_us.0 <= state.latest_timestamp {  
    return Err(ProgramError::AccountAlreadyInitialized);  
}
```

The example updates `state.latest_timestamp` only after this comparison succeeds. Nest's refresh paths perform the preceding message verification but omit this consumer-side ordering invariant, which is what permits an authentic yet older Lazer message to roll the snapshot back.

Snapshot consumers repeat only the feed and current-time policy checks and then use the resulting value:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/helpers/oracle.rs#L46-L74>

`mint_nusd_with_oracle` passes that snapshot-derived collateral value directly into `borrow`:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/ix/cdp.rs#L143-L176>

The attack can proceed as follows:

1. At $t = 1000$, the oracle publishes a valid collateral price of \$100.
2. At $t = 1050$, it publishes \$40, and the shared snapshot is refreshed to this newer value.
3. At $t = 1100$, the attacker resubmits the signed \$100 observation. It is 100 seconds old and therefore still valid under a 120-second freshness policy.
4. The refresh overwrites the \$40 snapshot with the older \$100 snapshot.
5. In the same transaction, the attacker calls `mint_nusd_with_oracle`. At a 45% borrow LTV, one token is treated as supporting \$45 of debt even though the newer price values it at only \$40.

The issue is distinct from failing to preserve a signed payload's `expires_at`: this rollback remains possible even when every expiry is honored because the missing invariant is ordering between two individually valid observations.

Impact

An attacker can over-mint nUSD or withdraw excess collateral during a sufficiently fast price decline or oracle correction. In the demonstrated case, replaying the \$100 observation permits \$45 of borrowing against collateral whose newer value is \$40. Even if all collateral is recovered, the position begins with a gross shortfall of at least \$5, equal to 11.1% of the minted debt, before liquidation costs.

The attack can be repeated across attacker-controlled vaults up to the applicable per-vault, collateral, and protocol debt caps. It can therefore create protocol bad debt and dilute the backing of circulating nUSD. Rolled-back prices also distort instant and two-step liquidation eligibility and collateral valuation. Impact is High; likelihood is Medium because exploitation requires retaining an older valid message, a favorable market move within the bounded freshness window, and timely transaction execution.

Code Snippet

The following verified domain-level PoC demonstrates that both the older \$100 observation and the newer \$40 observation pass the implemented freshness policy at $t = 1100$, while a \$45 borrow succeeds only with the older observation. The unconditional assignments shown above establish that the on-chain refresh paths can restore that older observation to the shared snapshot.

```
use nest_domain::{
    borrow, collateral_value_usd, lower_confidence_bound, CollateralParams,
    ↪ NestError,
    OraclePrice, PricePolicy, ProtocolAccounting, Vault, PRICE_SCALE,
};

fn feed() -> [u8; 32] {
    [7_u8; 32]
}

fn price(dollars: u64, publish_time: i64) -> OraclePrice {
    OraclePrice {
        feed_id: feed(),
        price_e8: dollars * PRICE_SCALE as u64,
        confidence_e8: 0,
        publish_time,
    }
}

fn policy() -> PricePolicy {
    PricePolicy {
```

```

        expected_feed_id: feed(),
        max_staleness_seconds: 120,
        max_confidence_bps: 200,
    }
}

fn params() -> CollateralParams {
    CollateralParams {
        borrow_ltv_bps: 4_500,
        liquidation_threshold_bps: 5_500,
        per_vault_debt_cap: u128::MAX,
        protocol_debt_cap: u128::MAX,
        collateral_debt_cap: u128::MAX,
        ..CollateralParams::default()
    }
}

#[test]
fn older_still_fresh_price_enables_borrow_rejected_by_newer_price() {
    let now = 1_100;
    let older_high = price(100, 1_000);
    let newer_low = price(40, 1_050);

    let old_safe_price = lower_confidence_bound(older_high, policy(), now).unwrap();
    let new_safe_price = lower_confidence_bound(newer_low, policy(), now).unwrap();
    assert_eq!(old_safe_price, 100 * PRICE_SCALE);
    assert_eq!(new_safe_price, 40 * PRICE_SCALE);

    let one_token_raw = 1_000_000_u128;
    let old_value = collateral_value_usd(one_token_raw, 6, old_safe_price).unwrap();
    let new_value = collateral_value_usd(one_token_raw, 6, new_safe_price).unwrap();
    assert_eq!(old_value, 100_000_000);
    assert_eq!(new_value, 40_000_000);

    let mut old_vault = Vault {
        collateral_raw: one_token_raw,
        ..Vault::default()
    };
    let mut old_protocol = ProtocolAccounting::new();
    borrow(
        &mut old_vault,
        &mut old_protocol,
        old_value,
        params(),
        45_000_000,
    )
    .unwrap();
    assert_eq!(old_vault.principal_debt, 45_000_000);

    let mut new_vault = Vault {

```

```

        collateral_raw: one_token_raw,
        ..Vault::default()
    };
    let mut new_protocol = ProtocolAccounting::new();
    assert_eq!(
        borrow(
            &mut new_vault,
            &mut new_protocol,
            new_value,
            params(),
            45_000_000,
        ),
        Err(NestError::InsufficientCollateral)
    );
}

```

Run from the contracts directory:

```
cargo test -p nest-domain --test oracle_rollback_poc -- --nocapture
```

Verified result:

```

running 1 test
test older_still_fresh_price_enables_borrow_rejected_by_newer_price ... ok
test result: ok. 1 passed; 0 failed

```

Tool Used

Manual Review

Recommendation

Enforce snapshot timestamp monotonicity in both `refresh_lazer_oracle` and `refresh_signed_oracle` before modifying `OracleSnapshot`. For an initialized snapshot, require the incoming `publish_time` to be strictly greater than `oracle.xstock_usd.publish_time`; reject older observations and same-timestamp observations that could contain conflicting values.

Apply this check in shared refresh logic so both price sources enforce the same invariant. Add regression tests that first store a newer observation and then attempt to submit an older but still-fresh observation through each refresh path, asserting that the second refresh fails and the newer snapshot remains unchanged.

Discussion

ducksquadd

<https://github.com/NestUSD/contracts/commit/1c6be41ae0c6eea9b6d07e84a7744632b4762267>

0502lian

Fixed

Issue M-1: PSM exits remain redeemable at par while protocol bad debt is outstanding [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/issues/55>

Summary

`psm_swap_out` lets any holder of the protocol `nUSD` mint burn `nUSD` and receive idle PSM USD C at 1:1 while the protocol has outstanding bad debt. Since CDP bad debt is written off in accounting without burning the corresponding circulating `nUSD`, underbacked `nUSD` remains fungible with PSM-minted `nUSD`. Fast redeemers can therefore extract PSM liquidity at par and push insolvency losses onto later holders.

Vulnerability Detail

When a user exits through the PSM, the account constraints only require the user's token account to hold the protocol `nUSD` mint. There is no distinction between `nUSD` minted through the PSM and `nUSD` minted by CDP borrowing:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/accounts/psm.rs#L23-L35>

```
#[derive(Accounts)]
pub struct PsmSwapOut<'info> {
    #[account(mut, seeds = [b"protocol"], bump = protocol.bump)]
    pub protocol: Box<Account<'info, Protocol>>,
    #[account(mut, address = protocol.usdc_mint)]
    pub usdc_mint: InterfaceAccount<'info, Mint>,
    #[account(mut, address = protocol.nusd_mint)]
    pub nusd_mint: InterfaceAccount<'info, Mint>,
    #[account(mut, token::mint = nusd_mint, token::authority = user,
        ↪ token::token_program = nusd_token_program)]
    pub user_nusd_account: InterfaceAccount<'info, TokenAccount>,
    #[account(mut, address = protocol.psm_usdc_vault, token::mint = usdc_mint,
        ↪ token::authority = protocol, token::token_program = usdc_token_program)]
    pub psm_usdc_vault: InterfaceAccount<'info, TokenAccount>,
    #[account(mut, token::mint = usdc_mint, token::authority = user,
        ↪ token::token_program = usdc_token_program)]
    pub user_usdc_account: InterfaceAccount<'info, TokenAccount>,
```

The exit instruction checks only the amount, global pause state, idle PSM liquidity, and the user's `nUSD` balance before reducing PSM accounting, burning the user's `nUSD`, and transferring the same amount of USDC:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/ix/psm.rs#L67-L99>

```

pub fn psm_swap_out(ctx: Context<PsmSwapOut>, amount: u64) -> Result<()> {
  require!(amount > 0, CoreError::InvalidParameter);
  require!(!ctx.accounts.protocol.paused, CoreError::Paused);
  let amount_u128 = amount as u128;
  let now = Clock::get()?.unix_timestamp;
  let user_nusd_before = ctx.accounts.user_nusd_account.amount;
  let nusd_supply_before = ctx.accounts.nusd_mint.supply;
  let protocol = &mut ctx.accounts.protocol;
  require!(
    protocol.psm_idle_usdc >= amount_u128,
    CoreError::PsmInsufficientLiquidity
  );
  require!(
    ctx.accounts.user_nusd_account.amount >= amount,
    CoreError::PsmInsufficientLiquidity
  );
  let psm_idle_before = protocol.psm_idle_usdc;
  let outflow_allowed = record_psm_outflow_or_pause(protocol, amount_u128,
    ↪ psm_idle_before, now)?;
  if !outflow_allowed {
    return Ok(());
  }
  protocol.psm_usdc_liabilities = protocol
    .psm_usdc_liabilities
    .checked_sub(amount_u128)
    .ok_or(error!(CoreError::MathOverflow))?;
  protocol.psm_nusd_supply = protocol
    .psm_nusd_supply
    .checked_sub(amount_u128)
    .ok_or(error!(CoreError::MathOverflow))?;
  protocol.psm_idle_usdc = protocol
    .psm_idle_usdc
    .checked_sub(amount_u128)
    .ok_or(error!(CoreError::MathOverflow))?;
}

```

There is no `protocol.bad_debt_nusd == 0` guard in this path.

This matters because full liquidation can close an underwater CDP by writing off uncovered debt and recording it as protocol bad debt:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/crates/nest-domain/src/cdp.rs#L448-L459>

```

if full && exhausts_collateral && vault.collateral_raw == 0 {
  let debt_after_repay = vault.total_debt()?;
  insurance_nusd_burned = core::cmp::min(protocol.insurance_fund_nusd,
    ↪ debt_after_repay);
  protocol.insurance_fund_nusd =
    checked_sub(protocol.insurance_fund_nusd, insurance_nusd_burned)?;
}

```

```

write_off_debt(vault, protocol, insurance_nusd_burned)?;

let uncovered = vault.total_debt()?;
if uncovered > 0 {
    write_off_debt(vault, protocol, uncovered)?;
    protocol.bad_debt_nusd = checked_add(protocol.bad_debt_nusd, uncovered)?;
    bad_debt = uncovered;
}
}

```

The Anchor liquidation instruction then burns only repaid principal plus insurance coverage. The uncovered bad debt is deliberately excluded from the burn amount:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/ix/liquidation/instant.rs#L69-L73>

```

let nusd_burned_u64 = u128_to_u64(
    out.principal_paid
        .checked_add(out.insurance_nusd_burned)
        .ok_or(error!(CoreError::MathOverflow))?,
)?;

```

So after an underwater CDP is liquidated, the vault debt can be written down to zero and `protocol.bad_debt_nusd` can increase, while the bad-debt portion of borrowed nUSD remains in circulation. Since all nUSD is the same mint, a holder of that underbacked nUSD can call `psm_swap_out` and receive idle PSM USDC at par.

The optional outflow breaker is not a sufficient protection. If disabled, it immediately returns `Ok(true)`:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/helpers/psm.rs#L26-L34>

```

fn record_psm_outflow_or_pause(
    protocol: &mut Protocol,
    amount: u128,
    liquid_basis: u128,
    now: i64,
) -> Result<bool> {
    if !protocol.psm_outflow_circuit_breaker_enabled {
        return Ok(true);
    }
}

```

It is disabled by default during protocol initialization:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/ix/setup.rs#L112-L118>

```

protocol.psm_outflow_window_start_ts = 0;
protocol.psm_outflow_window_usdc = 0;

```

```
protocol.psm_outflow_limit_usdc = 0;
protocol.psm_outflow_limit_bps = 0;
protocol.psm_outflow_window_seconds = SECONDS_PER_DAY;
protocol.psm_outflow_circuit_breaker_enabled = false;
protocol.psm_outflow_window_basis_usdc = 0;
```

Other PSM value-extraction paths already recognize that bad debt must be resolved first. For example, PSM yield routing and buyback surplus release both require `bad_debt_nusd == 0`, but `psm_swap_out` does not:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/helpers/revenue.rs#L11-L13>

```
require!(amount > 0, CoreError::InvalidParameter);
require!(!protocol.paused, CoreError::Paused);
require!(protocol.bad_debt_nusd == 0, CoreError::BadDebtOutstanding);
```

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/ix/liquidation/buyback.rs#L8-L13>

```
require!(amount > 0, CoreError::InvalidParameter);
require!(!ctx.accounts.protocol.paused, CoreError::Paused);
require!(
    ctx.accounts.protocol.bad_debt_nusd == 0,
    CoreError::BadDebtOutstanding
);
```

A concrete flow is:

1. The PSM has idle USDC and corresponding PSM nUSD liabilities.
2. A CDP borrower mints nUSD through `mint_nusd_with_oracle`.
3. The collateral becomes insufficient, and full liquidation writes off the uncovered debt into `protocol.bad_debt_nusd`.
4. The bad-debt portion of nUSD is not burned from circulating supply.
5. Any holder of this now-underbacked fungible nUSD calls `psm_swap_out`.
6. The instruction burns the holder's nUSD and transfers idle PSM USDC at 1:1 while bad debt is still outstanding.

Impact

Idle PSM USDC, and any later PSM liquidity added while bad debt remains outstanding, can be drained by holders of underbacked nUSD. This creates a first-exit priority during insolvency: early redeemers receive full par value, while later nUSD holders are left with the remaining protocol insolvency. The issue breaks proportional loss sharing and can materially worsen losses for users who do not redeem first.

Code Snippet

The existing domain test confirms that full liquidation can record bad debt:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/crates/nest-domain/src/tests.rs#L768-L805>

```
#[test]
fn full_liquidation_records_bad_debt_when_repay_exactly_exhausts_collateral() {
    let mut protocol = ProtocolAccounting::new();
    let mut vault = Vault {
        collateral_raw: 1_080_000,
        principal_debt: 200_000_000,
        accrued_fee: 0,
        last_accrual_ts: 0,
    };
    protocol.total_debt = vault.total_debt().unwrap();

    let out = liquidate(
        &mut vault,
        &mut protocol,
        108_000_000,
        100 * PRICE_SCALE,
        6,
        params,
        100_000_000,
    )
    .unwrap();

    assert_eq!(out.bad_debt, 100_000_000);
    assert_eq!(protocol.bad_debt_nusd, 100_000_000);
    assert_eq!(protocol.total_debt, 0);
}
```

The core PSM outflow breaker test confirms the breaker only pauses after configured thresholds when enabled; it does not add a bad-debt solvency check to `psm_swap_out`.

Tool Used

Manual Review

```
cargo test -p nest-domain full_liquidation_records_bad_debt_when_repay_exactly_exhausts_collateral --manifest-path contracts/Cargo.toml
```

```
cargo test -p nest-core psm_outflow_breaker_allows_exact_limit_and_pauses_on_excess --manifest-path contracts/Cargo.toml
```

Recommendation

Block PSM exits while protocol bad debt is outstanding:

```
require!(  
  ctx.accounts.protocol.bad_debt_nusd == 0,  
  CoreError::BadDebtOutstanding  
);
```

This check should run before any PSM outflow accounting mutation or token transfer in `psm_swap_out`.

If redemptions during bad debt are an intended feature, implement an explicit insolvency-mode redemption mechanism instead of paying full par. For example, apply a proportional haircut based on total outstanding `nUSD`, PSM liquidity, CDP collateral backing, and `bad_debt_nusd`, so losses are shared fairly instead of allocated by transaction ordering.

Discussion

ducksquadd

<https://github.com/NestUSD/contracts/commit/f93912253a136400a16ad3657faa344c7506717b>

0502lian

Fixed

Issue M-2: Bad debt cover can consume PSM surplus for unminted accrued fees [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/issues/59>

Summary

Full liquidation records the remaining `vault.total_debt()` as `protocol.bad_debt_nusd`. However, `total_debt()` is `principal_debt + accrued_fee`, while only principal debt was minted as circulating `nUSD` when the user borrowed. Stability fees are only accounting receivables until they are actually paid.

As a result, unpaid accrued fees can be mixed into `bad_debt_nusd`. When `cover_bad_debt` later covers that amount, it consumes real PSM `USDC` surplus and increases PSM liabilities for debt that never corresponded to minted `nUSD`.

Vulnerability Detail

Borrowing increases vault principal and mints exactly the borrowed amount of `nUSD` to the borrower:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/ix/cdp.rs#L190-L198>

```
token_mint_to_checked(
    ctx.accounts.nusd_token_program.to_account_info(),
    ctx.accounts.nusd_mint.to_account_info(),
    ctx.accounts.owner_nusd_account.to_account_info(),
    ctx.accounts.protocol.to_account_info(),
    amount,
    ctx.accounts.nusd_mint.decimals,
    signer_seeds,
)?;
```

Stability fee accrual does not mint any `nUSD`. It only increases the vault's fee receivable and protocol accounting debt:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/crates/nest-domain/src/cdp.rs#L186-L189>

```
vault.accrued_fee = checked_add(vault.accrued_fee, fee)?;
protocol.total_debt = checked_add(protocol.total_debt, fee)?;
protocol.total_uncollected_fees = checked_add(protocol.total_uncollected_fees,
    ↪ fee)?;
vault.last_accrual_ts = now;
```

The vault total debt includes both components:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/crates/nest-domain/src/cdp.rs#L126-L129>

```
impl Vault {
    pub fn total_debt(&self) -> Result<u128> {
        checked_add(self.principal_debt, self.accrued_fee)
    }
}
```

When debt is actually repaid, the accounting correctly treats fees and principal differently. Fees are paid first, routed as realized revenue, and not burned. Only the remaining amount repays principal:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/crates/nest-domain/src/cdp.rs#L296-L305>

```
let fee_paid = core::cmp::min(amount, vault.accrued_fee);
vault.accrued_fee = checked_sub(vault.accrued_fee, fee_paid)?;
protocol.total_uncollected_fees = checked_sub(protocol.total_uncollected_fees,
    ↪ fee_paid)?;
protocol.total_debt = checked_sub(protocol.total_debt, fee_paid)?;
protocol.route_realized_fee(fee_paid)?;

let remaining = checked_sub(amount, fee_paid)?;
let principal_paid = core::cmp::min(remaining, vault.principal_debt);
vault.principal_debt = checked_sub(vault.principal_debt, principal_paid)?;
protocol.total_debt = checked_sub(protocol.total_debt, principal_paid)?;
```

The Anchor liquidation path mirrors that split at the token layer. It transfers the routed fee and liquidation penalty to revenue vaults, but burns only principal_paid plus insurance coverage:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/ix/liquidation/instant.rs#L63-L73>

```
let liquidator_paid_u64 = u128_to_u64(
    out.fee_paid
        .checked_add(out.principal_paid)
        .and_then(|amount| amount.checked_add(out.staker_penalty_nusd))
        .ok_or(error!(CoreError::MathOverflow))?,
)?;
let nusd_burned_u64 = u128_to_u64(
    out.principal_paid
        .checked_add(out.insurance_nusd_burned)
        .ok_or(error!(CoreError::MathOverflow))?,
)?;
```

The bug occurs when full liquidation exhausts collateral. After applying the liquidator

repayment and any insurance burn, the remaining `vault.total_debt()` is recorded as bad debt:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/crates/nest-domain/src/cdp.rs#L448-L459>

```
if full && exhausts_collateral && vault.collateral_raw == 0 {
    let debt_after_repay = vault.total_debt()?;
    insurance_nusd_burned = core::cmp::min(protocol.insurance_fund_nusd,
        ↪ debt_after_repay);
    protocol.insurance_fund_nusd =
        checked_sub(protocol.insurance_fund_nusd, insurance_nusd_burned)?;
    write_off_debt(vault, protocol, insurance_nusd_burned)?;

    let uncovered = vault.total_debt()?;
    if uncovered > 0 {
        write_off_debt(vault, protocol, uncovered)?;
        protocol.bad_debt_nusd = checked_add(protocol.bad_debt_nusd, uncovered)?;
        bad_debt = uncovered;
    }
}
```

Because `uncovered` is computed from `total_debt()`, it can include unpaid accrued fees. Those fees were never minted as `nUSD`; they are expected future revenue. `write_off_debt` then cancels the unpaid fee receivable, but the pre-writeoff `uncovered` amount is still added to `bad_debt_nusd`:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/crates/nest-domain/src/cdp.rs#L315-L334>

```
fn write_off_debt(
    vault: &mut Vault,
    protocol: &mut ProtocolAccounting,
    amount: u128,
) -> Result<u128> {
    ...
    let fee_cancelled = core::cmp::min(writeoff, vault.accrued_fee);
    vault.accrued_fee = checked_sub(vault.accrued_fee, fee_cancelled)?;
    protocol.total_uncollected_fees = checked_sub(protocol.total_uncollected_fees,
        ↪ fee_cancelled)?;
    protocol.total_debt = checked_sub(protocol.total_debt, fee_cancelled)?;

    let remaining = checked_sub(writeoff, fee_cancelled)?;
    let principal_cancelled = core::cmp::min(remaining, vault.principal_debt);
    vault.principal_debt = checked_sub(vault.principal_debt, principal_cancelled)?;
    protocol.total_debt = checked_sub(protocol.total_debt, principal_cancelled)?;
    checked_add(fee_cancelled, principal_cancelled)
}
```

Later, `cover_bad_debt` requires real surplus `USDC` in the PSM vault and increases PSM

liabilities and PSM nUSD supply by the full covered amount:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/ix/revenue.rs#L102-L136>

```
let recorded_idle = ctx.accounts.protocol.psm_idle_usdc;
let actual_idle = ctx.accounts.psm_usdc_vault.amount as u128;
require!(
    actual_idle
        >= recorded_idle
            .checked_add(amount_u128)
            .ok_or(error!(CoreError::MathOverflow))?,
    CoreError::PsmNoSurplus
);
let new_liabilities = ctx
    .accounts
    .protocol
    .psm_usdc_liabilities
    .checked_add(amount_u128)
    .ok_or(error!(CoreError::MathOverflow))?;
...
ctx.accounts.protocol.psm_usdc_liabilities = new_liabilities;
ctx.accounts.protocol.psm_nusd_supply = ctx
    .accounts
    .protocol
    .psm_nusd_supply
    .checked_add(amount_u128)
    .ok_or(error!(CoreError::MathOverflow))?;
ctx.accounts.protocol.psm_idle_usdc = recorded_idle
    .checked_add(amount_u128)
    .ok_or(error!(CoreError::MathOverflow));
```

This is appropriate for uncovered principal, because borrowed principal was minted as nUSD and can remain in circulation. It is not appropriate for unpaid fees, because no corresponding nUSD supply was minted for those fees.

A concrete flow is:

1. A vault has `principal_debt = 100` and `accrued_fee = 10`.
2. Only the 100 principal was minted to the borrower when debt was created.
3. Full liquidation exhausts collateral and repays only 1 nUSD.
4. Repayment applies to fees first, leaving `principal_debt = 100` and `accrued_fee = 9`.
5. The full liquidation branch computes `uncovered = vault.total_debt() = 109`.
6. `write_off_debt` cancels the 9 unpaid fee and the 100 principal, but `bad_debt_nusd` is increased by the full 109.

7. `cover_bad_debt(109)` later consumes 109 real PSM surplus and creates 109 PSM liabilities, even though only 100 corresponds to minted principal nUSD.

Impact

Protocol surplus can be unnecessarily locked or diverted into covering non-existent PSM liabilities. The inflated `bad_debt_nusd` also blocks PSM surplus routing and buyback release until the overstated amount is covered:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/helpers/revenue.rs#L11-L13>

```
require!(amount > 0, CoreError::InvalidParameter);
require!(!protocol.paused, CoreError::Paused);
require!(protocol.bad_debt_nusd == 0, CoreError::BadDebtOutstanding);
```

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/ix/liquidation/buyback.rs#L8-L13>

```
require!(amount > 0, CoreError::InvalidParameter);
require!(!ctx.accounts.protocol.paused, CoreError::Paused);
require!(
    ctx.accounts.protocol.bad_debt_nusd == 0,
    CoreError::BadDebtOutstanding
);
```

The unpaid fee portion is not paid to insurance, stakers, or protocol revenue. It is cancelled as an uncollected receivable, but then incorrectly transformed into PSM bad debt cover demand. This can consume real USDC surplus that should remain available for valid PSM backing, yield routing, or buyback release.

Code Snippet

The vulnerable accounting is the addition of full uncovered total debt to `bad_debt_nusd`:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/crates/nest-domain/src/cdp.rs#L455-L459>

That value can include `vault.accrued_fee`:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/crates/nest-domain/src/cdp.rs#L126-L129>

Tool Used

Manual Review

Recommendation

Do not add cancelled accrued fees to `protocol.bad_debt_nusd`. Bad debt that must later be covered by PSM surplus should be limited to the uncovered principal amount, because that is the portion that corresponds to previously minted nUSD.

Refactor the writeoff logic to return separate `fee_cancelled` and `principal_cancelled` values, then increment `bad_debt_nusd` only by the uncovered principal cancellation. Unpaid accrued fees should only reduce `vault.accrued_fee`, `protocol.total_uncollected_fees`, and `protocol.total_debt`, representing lost revenue rather than a circulating nUSD supply deficit.

Discussion

ducksquaddd

https://github.com/NestUSD/contracts/commit/3393306baa5cca95891624cf55d99db_e3276c863

0502lian

Fixed

Issue M-3: Two-step liquidation settlement can deadlock when surplus exists during bad debt [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/issues/60>

Summary

`settle_liquidation_proceeds()` can become impossible to execute when a two-step liquidation receipt requires settlement above principal while `protocol.bad_debt_nusd > 0`. The settlement floor forces the liquidator to pay the recorded collateral value, but any amount above principal is treated as PSM yield and routed through `apply_realized_psm_yield_accounting()`, which reverts while bad debt exists.

Vulnerability Detail

In two-step liquidation settlement, the required payment is:

https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/ix/liquidation/two_step.rs#L137

```
max(principal_debt, min_settlement_usdc)
```

So if the recorded collateral value was higher than the principal debt, the liquidator must settle with surplus:

https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/ix/liquidation/two_step.rs#L134-144

```
let principal_debt = ctx.accounts.liquidation_receipt.principal_debt;
let min_settlement_usdc = ctx.accounts.liquidation_receipt.min_settlement_usdc;
let settlement_floor = core::cmp::max(principal_debt, min_settlement_usdc);

require!(
    usdc_amount as u128 >= settlement_floor,
    CoreError::InsufficientLiquidationProceeds
);
```

Later, the settlement logic treats all USDC above principal as revenue:

https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/ix/liquidation/two_step.rs#L206-221

```
let revenue_amount = (usdc_amount as u128)
    .checked_sub(principal_debt)
    .ok_or(error!(CoreError::MathOverflow))?;

if revenue_amount > 0 {
```

```

    apply_realized_psm_yield_accounting(
        &mut ctx.accounts.protocol,
        ctx.accounts.psm_usdc_vault.amount,
        u128_to_u64(revenue_amount)?,
        staking_assets,
        Clock::get()?.unix_timestamp,
    );
}

```

However, `apply_realized_psm_yield_accounting()` rejects if the protocol has outstanding bad debt or is paused:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/helpers/revenue.rs#L11-31>

```

require!(amount > 0, CoreError::InvalidParameter);
require!(!protocol.paused, CoreError::Paused);
require!(protocol.bad_debt_nusd == 0, CoreError::BadDebtOutstanding);

let new_liabilities = protocol
    .psm_usdc_liabilities
    .checked_add(amount_u128)
    .ok_or(error!(CoreError::MathOverflow))?;

require!(
    new_liabilities <= protocol.psm_cap,
    CoreError::PsmCapExceeded
);

```

This creates a deadlock situation: imagine a two-step liquidation receipt requires surplus settlement because `min_settlement_usdc > principal_debt`. Here, the liquidator cannot settle below `min_settlement_usdc`. Paying the required surplus enters the PSM yield path and the PSM yield path reverts if `bad_debt_nusd > 0`. The settlement USDC that could create surplus for bad-debt coverage never enters the protocol.

Impact

A valid two-step liquidation receipt can become permanently unsettlable while bad debt exists. During this state, the liquidation principal remains in `pending_liquidation_principal`, collateral has already been seized, and the protocol cannot finalize the receipt or recognize the settlement USDC. This can also lock liquidation accounting and worsen bad-debt recovery.

Tool Used

Manual Review

Recommendation

Consider not routing two-step liquidation surplus through the normal PSM yield path before settlement is finalized. Additionally, settlement finalization should not be blocked by paused state when the payment is needed to resolve an already-started liquidation.

Discussion

ducksquadd

<https://github.com/NestUSD/contracts/commit/5154b6b04ee7cba199511819421707eaf176f43b>

fixed this

0502lian

Fixed

Issue M-4: CDP fee and liquidation revenue can be distributed while bad debt exists [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/issues/64>

Summary

The protocol blocks PSM yield distribution while `bad_debt_nusd > 0`, but CDP fee repayment and instant liquidation revenue don't enforce the same bad-debt seniority. Fees and liquidation charges can be transferred to the insurance, staker-revenue, and protocol-revenue vaults while the system still has outstanding bad debt.

Vulnerability Detail

The PSM yield path explicitly rejects revenue distribution while bad debt exists:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/helpers/revenue.rs#L11-13>

```
require!(amount > 0, CoreError::InvalidParameter);
require!(!protocol.paused, CoreError::Paused);
require!(protocol.bad_debt_nusd == 0, CoreError::BadDebtOutstanding);
```

However, CDP repayment routes accrued fees through `route_realized_fee()` with no equivalent `bad_debt_nusd` check:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/crates/nest-domain/src/cdp.rs#L296-300>

```
let fee_paid = core::cmp::min(amount, vault.accrued_fee);
vault.accrued_fee = checked_sub(vault.accrued_fee, fee_paid)?;
protocol.total_uncollected_fees = checked_sub(protocol.total_uncollected_fees,
    ↪ fee_paid)?;
protocol.total_debt = checked_sub(protocol.total_debt, fee_paid)?;
protocol.route_realized_fee(fee_paid)?;
```

Then `repay_nusd()` transfers the routed fee portions to the insurance, staker-revenue, and protocol-revenue vaults, while only the principal portion is burned:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/ix/cdp.rs#L255-265>

```
let (staker_delta, protocol_delta) = route_targeted_staker_revenue(
    &mut ctx.accounts.protocol,
    staking_assets,
    now,
    staker_delta,
```

```
)?;  
  
let routed_fee = insurance_delta  
  .checked_add(staker_delta)  
  .and_then(|v| v.checked_add(protocol_delta))  
  .ok_or(error!(CoreError::MathOverflow))?;  
require!(routed_fee == out.fee_paid, CoreError::MathOverflow);
```

The instant liquidation path has the same issue. It routes `fee_paid + staker_penalty_nus` to revenue vaults, but burns only principal and insurance coverage. This means the protocol can distribute value as revenue even though bad debt should be senior to all revenue.

Impact

Value that should be used to reduce bad debt can instead be routed to revenue recipients. Stakers can later extract staker revenue because `harvest()` only checks the revenue vault balance and has no bad-debt gate.

Tool Used

Manual Review

Recommendation

Make bad debt senior to all CDP fee and liquidation revenue.

Discussion

ducksquadd

<https://github.com/NestUSD/contracts/commit/b6a163d328231c9d3b083d6c352643bfcfbb593f>

0502lian

Fixed

Issue M-5: PSM circuit breaker can pause protocol through successful no-op redemption [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/issues/65>

Summary

`psm_swap_out()` can successfully return without burning the caller's nUSD or transferring USDC, while still committing `protocol.paused = true`. A user can intentionally request a PSM redemption above the configured outflow limit to pause the whole protocol at no asset cost.

Vulnerability Detail

The PSM outflow circuit breaker is implemented in `record_psm_outflow_or_pause()`. When the next outflow would exceed the active limit, it sets the protocol pause flag and returns `Ok(false)`:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/helpers/psm.rs#L77-80>

```
if next_window_usdc > active_limit {
    protocol.paused = true;
    return Ok(false);
}
```

`psm_swap_out()` then treats `false` as a successful no-op:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/ix/psm.rs#L84-87>

```
let outflow_allowed = record_psm_outflow_or_pause(protocol, amount_u128,
    ↪ psm_idle_before, now)?;
if !outflow_allowed {
    return Ok(());
}
```

Because the instruction returns `Ok()`, the pause state is committed. However, the burn and USDC transfer happen only after this block:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/ix/psm.rs#L100-108>

```
token_burn_checked(
    ctx.accounts.nusd_token_program.to_account_info(),
    ctx.accounts.nusd_mint.to_account_info(),
    ctx.accounts.user_nusd_account.to_account_info(),
```

```
ctx.accounts.user.to_account_info(),  
amount,  
ctx.accounts.nusd_mint.decimals,  
&[],  
)?;
```

So when the breaker trips, the caller keeps their nUSD, receives no USDC, and still pauses the protocol.

Impact

Any user who holds enough nUSD to request an over-limit redemption can pause the protocol for free, provided the circuit breaker is enabled and the PSM has enough idle USDC for the requested amount. The attack is repeatable and recovery requires an admin to call `set_paused(false)`. This can halt protocol functionality such as borrowing, PSM operations, Kamino flows, and buyback operations.

Tool Used

Manual Review

Recommendation

Consider not silently returning success from `psm_swap_out()` when the circuit breaker is exceeded. Either revert with a circuit-breaker error, or move the pause action into a separate explicit instruction.

Discussion

ducksquadd

This is intended behavior at least until audits are finished and what not. Just as a safety mechanism.

ducksquadd

<https://github.com/NestUSD/contracts/commit/c086d14085d0ea3ed44c8c0d1bc6d05fa895bdfc>

0502lian

Fixed

Issue M-6: Two-step settlement can be blocked because PSM capacity is not reserved [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/issues/67>

Summary

Two-step liquidation start moves debt into `pending_liquidation_principal` without reserving PSM capacity. Settlement later adds the same principal to `psm_usdc_liabilities` and checks it against `psm_cap`. Because regular `psm_swap_in` can fill the PSM cap before settlement, a valid liquidation receipt can become temporarily unseizable with `PsmCapExceeded`.

Vulnerability Detail

At liquidation start, the protocol records the pending liquidation principal but does not check or reserve PSM capacity:

https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/ix/liquidation/two_step.rs#L65-70

```
ctx.accounts.protocol.pending_liquidation_principal = ctx
    .accounts
    .protocol
    .pending_liquidation_principal
    .checked_add(principal_debt)
    .ok_or(error!(CoreError::MathOverflow))?;
```

Later, during settlement, the protocol adds the principal to PSM liabilities and only then enforces the cap:

https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/ix/liquidation/two_step.rs#L183-192

```
ctx.accounts.protocol.psm_usdc_liabilities = ctx
    .accounts
    .protocol
    .psm_usdc_liabilities
    .checked_add(principal_debt)
    .ok_or(error!(CoreError::MathOverflow))?;

require!(
    ctx.accounts.protocol.psm_usdc_liabilities <= ctx.accounts.protocol.psm_cap,
    CoreError::PsmCapExceeded
);
```


This creates an asymmetry: the protocol starts liquidation and transfers collateral without reserving room for the later settlement, but settlement can fail if the PSM cap becomes full before it is called.

An ordinary user can trigger this by using `psm_swap_in`, which is permissionless and also limited by the same `psm_cap`. Example:

```
PSM cap:                1,000
Current PSM liabilities: 900
Two-step liquidation debt: 100
```

Liquidation start succeeds and records 100 as pending. Before settlement, a user swaps in 100 USDC through the PSM:

```
PSM liabilities: 1,000
```

Now settlement tries to add the pending 100:

```
1,000 + 100 > 1,000
```

and reverts with `PsmCapExceeded`.

Impact

A valid two-step liquidation receipt can become temporarily unseizable after collateral has already been transferred to the liquidation authority. This can delay liquidation finalization and leave principal stuck in `pending_liquidation_principal`.

Tool Used

Manual Review

Recommendation

Reserve PSM capacity when a two-step liquidation is started, or include `pending_liquidation_principal` in PSM cap checks.

Discussion

ducksquadd

<https://github.com/NestUSD/contracts/commit/5154b6b04ee7cba199511819421707eaf176f43b> Fixed this

0502lian

Fixed

Issue M-7: Liquidation ignores keeper/insurance penalty share constants [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/issues/68>

Summary

The protocol declares `KEEPER_PENALTY_SHARE_BPS = 6_000` and `INSURANCE_PENALTY_SHARE_BPS = 4_000`, which indicates that liquidation penalties may be intended to be divided 60/40 between the keeper and insurance. Neither constant is referenced anywhere else in the codebase. Independently, the protocol's active `route_realized_fee()` policy routes `insurance_fee_share_bps` of realized revenue to insurance while `insurance_fund_nusd` is below `cdp_insurance_target()`.

Instead, instant liquidation includes the entire penalty in the collateral value assigned to the liquidator, records the delivered penalty as `staker_penalty_nusd`, credits it directly to `realized_revenue_for_stakers`, and always returns `insurance_collateral_raw: 0`. The direct credit also bypasses `route_realized_fee()`, so the target-aware nUSD insurance allocation is skipped even when insurance is below target. The on-chain instruction supports transferring both insurance collateral and target-routed insurance nUSD, but neither branch receives any share of the instant-liquidation penalty. The two-step path also bypasses the declared fixed split: all seized collateral is initially transferred to the liquidator, while settlement proceeds above principal are accounted for as generic PSM yield.

There are therefore two possible policy interpretations, and they must not be applied cumulatively to the same penalty. If the liquidation-specific constants are authoritative, insurance misses its fixed 40% collateral share. If instant-liquidation penalties are realized protocol revenue, the active fee policy instead requires a target-aware `insurance_fee_share_bps` allocation (20% by default while insurance is below target). Existing tests explicitly assert the current zero-insurance behavior, so sponsor confirmation is still required to identify the authoritative economic policy; under either interpretation, however, the current instant path allocates no penalty value to insurance.

Vulnerability Detail

The liquidation constants define a complete 60/40 allocation of the penalty:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/crates/nest-domain/src/constants.rs#L8-L14>

```
pub const DEFAULT_LIQUIDATION_PENALTY_BPS: u16 = 800;
// ...
pub const KEEPER_PENALTY_SHARE_BPS: u16 = 6_000;
pub const INSURANCE_PENALTY_SHARE_BPS: u16 = 4_000;
```

A repository-wide reference search at revision `ced222f7c8f85f032f12cae006e978ea9a4f1b8` finds no use of either `share` constant outside these declarations.

The core `liquidate()` implementation computes the full penalty and adds all of it to `keeper_value`. That combined value is converted into the amount of collateral seized for the keeper/liquidator:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/crates/nest-domain/src/cdp.rs#L411-L420>

```
let penalty_value = mul_div_up(
    repay_amount,
    params.liquidation_penalty_bps as u128,
    BPS_DENOMINATOR,
)?;
let keeper_value = checked_add(repay_amount, penalty_value)?;

let keeper_raw =
    collateral_raw_for_usd_value(keeper_value, token_decimals,
    ↪ raw_token_safe_price_e8)?;
```

After limiting the seizure to available collateral, the function treats the entire delivered penalty as `staker_penalty_nusd` and adds it to realized staker revenue. No keeper/insurance split is applied:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/crates/nest-domain/src/cdp.rs#L432-L444>

```
let delivered_value =
    raw_collateral_value_usd(keeper_raw, token_decimals, raw_token_safe_price_e8)?;
let delivered_penalty_value = if delivered_value > repay_amount {
    checked_sub(delivered_value, repay_amount)?
} else {
    0
};
let staker_penalty_nusd = core::cmp::min(penalty_value, delivered_penalty_value);

let repay_outcome = repay(vault, protocol, repay_amount)?;
protocol.realized_revenue_for_stakers =
    checked_add(protocol.realized_revenue_for_stakers, staker_penalty_nusd)?;
vault.collateral_raw = checked_sub(vault.collateral_raw, keeper_raw)?;
```

This direct addition also skips the protocol's existing target-aware realized-fee router. `route_realized_fee()` computes a debt-relative insurance target and, while insurance is below it, allocates `insurance_fee_share_bps` of the realized amount to `insurance_fund_nusd`; only the remainder becomes staker revenue:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/crates/nest-domain/src/cdp.rs#L82-L106>

```
pub fn route_realized_fee(&mut self, amount: u128) -> Result<()> {
    let target = self.cdp_insurance_target()?;
    let insurance_share = if self.insurance_fund_nusd >= target {
        0
    } else {
        let desired = mul_div_down(
            amount,
            self.insurance_fee_share_bps as u128,
            BPS_DENOMINATOR,
        )?;
        core::cmp::min(desired, target - self.insurance_fund_nusd)
    };
    let staker_share = checked_sub(amount, insurance_share)?;
    self.insurance_fund_nusd = checked_add(self.insurance_fund_nusd,
        ↪ insurance_share)?;
    self.realized_revenue_for_stakers =
        checked_add(self.realized_revenue_for_stakers, staker_share)?;
    Ok(())
}
```

The ordinary CDP repayment path applies this policy to realized stability fees:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/crates/nest-domain/src/cdp.rs#L288-L305>

```
let fee_paid = core::cmp::min(amount, vault.accrued_fee);
// ... reduce accrued fee and total debt ...
protocol.route_realized_fee(fee_paid)?;
```

By contrast, `liquidate()` calls `repay()` and then bypasses the same router for `staker_penalty_nusd`. This is an independent indicator of missing insurance routing even if the unused 60/40 constants are obsolete. Under the default `insurance_fee_share_bps = 2_000`, a realized 2_400_000 penalty would route 480_000 nUSD to insurance while the fund has sufficient target capacity, leaving 1_920_000 as pre-targeted staker revenue.

The returned result then assigns all seized collateral to `keeper_collateral_raw` and hard-codes the insurance allocation to zero:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/crates/nest-domain/src/cdp.rs#L463-L470>

```
Ok(LiquidationOutcome {
    repaid_debt: repay_amount,
    fee_paid: repay_outcome.fee_paid,
    principal_paid: repay_outcome.principal_paid,
    staker_penalty_nusd,
    keeper_collateral_raw: keeper_raw,
    insurance_collateral_raw: 0,
```

```
insurance_nusd_burned,
```

The instant liquidation instruction charges the liquidator for the full `staker_penalty_nusd` amount in addition to fees and principal:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/ix/liquidation/instant.rs#L63-L68>

```
let liquidator_paid_u64 = u128_to_u64(
    out.fee_paid
        .checked_add(out.principal_paid)
        .and_then(|amount| amount.checked_add(out.staker_penalty_nusd))
        .ok_or(error!(CoreError::MathOverflow))?,
)?;
```

The instruction computes `insurance_delta` and `staker_delta` from the domain result, then sends `staker_delta` through a later staker/protocol revenue split. Because the penalty never passed through `route_realized_fee()`, `insurance_delta` contains no target-aware share of that penalty. `route_targeted_staker_revenue()` cannot repair the omission because it only subdivides revenue already classified for stakers:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/ix/liquidation/instant.rs#L51-L61>

```
let insurance_delta = insurance_after_before_burn
    .checked_sub(insurance_before)
    .ok_or(error!(CoreError::MathOverflow))?;
let staker_delta = protocol
    .realized_revenue_for_stakers
    .checked_sub(staker_before)
    .ok_or(error!(CoreError::MathOverflow))?;
```

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/ix/liquidation/instant.rs#L106-L123>

```
let (staker_delta, protocol_delta) = route_targeted_staker_revenue(
    &mut ctx.accounts.protocol,
    staking_assets,
    now,
    staker_delta,
)?;
let routed_charge = insurance_delta
    .checked_add(staker_delta)
    .and_then(|amount| amount.checked_add(protocol_delta))
    .ok_or(error!(CoreError::MathOverflow))?;
```

It transfers `keeper_collateral_raw` to the liquidator and contains a separate insurance-collateral transfer branch. However, that second branch only runs when `out.insurance_collateral_raw > 0`, which the domain function never emits:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/ix/liquidation/instant.rs#L211-L242>

```
let keeper_collateral = u128_to_u64(out.keeper_collateral_raw)?;
// ... transfer keeper_collateral to liquidator_collateral_account ...

let insurance_collateral = u128_to_u64(out.insurance_collateral_raw)?;
if insurance_collateral > 0 {
    token_transfer_checked(
        ctx.accounts.collateral_token_program.to_account_info(),
        ctx.accounts.collateral_vault.to_account_info(),
        ctx.accounts.collateral_mint.to_account_info(),
        ctx.accounts.insurance_collateral_account.to_account_info(),
        ctx.accounts.protocol.to_account_info(),
        insurance_collateral,
        ctx.accounts.collateral_mint.decimals,
        signer_seeds,
    );
}
```

The existing domain test demonstrates and blesses the resulting allocation. For a repayment of 30_000_000 atomic units and an 800 bps penalty, the total penalty is 2_400_000. Applying the declared split would produce a 1_440_000 keeper penalty and a 960_000 insurance penalty. The actual output instead assigns all 2_400_000 to staker_penalty_nusd and leaves insurance collateral at zero:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/crates/nest-domain/src/tests.rs#L555-L573>

```
let out = liquidate(
    // ...
    params,
    30_000_000,
)
.unwrap();

assert_eq!(out.repaid_debt, 30_000_000);
assert!(out.keeper_collateral_raw > 0);
assert_eq!(out.insurance_collateral_raw, 0);
assert_eq!(out.staker_penalty_nusd, 2_400_000);
assert_eq!(
    protocol.realized_revenue_for_stakers,
    out.staker_penalty_nusd
);
```

The two-step path does not restore the intended allocation. At liquidation start, it transfers the vault's entire seized collateral balance to the liquidator:

https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/ix/liquidation/two_step.rs#L94-L120

```

let collateral_seized = u128_to_u64(collateral_raw)?;
// ...
token_transfer_checked(
    ctx.accounts.collateral_token_program.to_account_info(),
    ctx.accounts.collateral_vault.to_account_info(),
    ctx.accounts.collateral_mint.to_account_info(),
    ctx.accounts.liquidator_collateral_account.to_account_info(),
    ctx.accounts.protocol.to_account_info(),
    collateral_seized,
    ctx.accounts.collateral_mint.decimals,
    signer_seeds,
)?;

```

When the liquidator settles, all USDC above principal is passed to the generic PSM yield accounting function rather than split with KEEPER_PENALTY_SHARE_BPS and INSURANCE_PENALTY_SHARE_BPS:

https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/ix/liquidation/two_step.rs#L206-L220

```

let revenue_amount = (usdc_amount as u128)
    .checked_sub(principal_debt)
    .ok_or(error!(CoreError::MathOverflow))?;
if revenue_amount > 0 {
    // ...
    let (insurance_delta_u64, staker_delta_u64, protocol_delta_u64,
        ↳ minted_nusd_u64) =
        apply_realized_psm_yield_accounting(
            &mut ctx.accounts.protocol,
            ctx.accounts.psm_usdc_vault.amount,
            u128_to_u64(revenue_amount)?,
            staking_assets,
            Clock::get()?.unix_timestamp,
        );
}

```

Consequently, neither liquidation mode applies the specifically declared liquidation-penalty allocation.

Impact

The exact shortfall depends on which of the two non-cumulative policy interpretations is authoritative.

If the 60/40 liquidation constants represent the intended economics, the insurance side is underfunded by 40% of every realized liquidation penalty. With the default 8% liquidation penalty, the omitted insurance allocation is:

```
8% liquidation penalty * 40% insurance share = 3.2% of repaid debt
```

For the concrete 30_000_000-unit repayment, this is 960_000 atomic units of penalty value that should be credited to insurance but is not. Across liquidation volume V at default parameters, the cumulative shortfall is approximately $0.032 * V$, weakening the reserve available to absorb future bad debt.

If the fixed constants are obsolete and instant-liquidation penalties should instead follow the active target-aware realized-fee policy, the default shortfall while insurance is below target is:

```
8% liquidation penalty * 20% insurance fee share = 1.6% of repaid debt
```

For the same 30_000_000 repayment, the realized penalty is 2_400_000; `route_realized_fee()` would allocate 480_000 to insurance while target capacity remains, but the current path allocates zero penalty revenue to insurance and credits all 2_400_000 directly as staker revenue. This shortfall is capped as the insurance fund approaches its configured target and becomes zero once the target is met.

This report does not claim that funds disappear or that a liquidator can directly extract the omitted insurance share. Under the current instant path, the value is instead classified and routed through `staker_penalty_nusd`, while two-step excess settlement is handled as PSM yield. The issue is an economic allocation mismatch that underfunds reserves available to absorb future bad debt. Sponsor confirmation is required to choose between the fixed 40% liquidation policy and the target-aware 20%-by-default realized-fee policy, but the two amounts should not be summed or simultaneously applied.

Code Snippet

The decisive inconsistency is that the protocol declares a 40% insurance share but the liquidation result can only return zero insurance collateral:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/crates/nest-domain/src/constants.rs#L13-L14>

```
pub const KEEPER_PENALTY_SHARE_BPS: u16 = 6_000;  
pub const INSURANCE_PENALTY_SHARE_BPS: u16 = 4_000;
```

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/crates/nest-domain/src/cdp.rs#L463-L469>

```
Ok(LiquidationOutcome {  
    // ...  
    staker_penalty_nusd,  
    keeper_collateral_raw: keeper_raw,
```



```
insurance_collateral_raw: 0,
```

The on-chain insurance transfer is therefore unreachable for penalty collateral:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/ix/liquidation/instant.rs#L232-L242>

```
let insurance_collateral = u128_to_u64(out.insurance_collateral_raw)?;
if insurance_collateral > 0 {
    token_transfer_checked(
        // ...
        ctx.accounts.insurance_collateral_account.to_account_info(),
        // ...
        insurance_collateral,
        // ...
    );
}
```

The same penalty is also added directly to staker revenue instead of going through the existing insurance-aware realized-fee policy:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/crates/nest-domain/src/cdp.rs#L439-L443>

```
let staker_penalty_nusd = core::cmp::min(penalty_value, delivered_penalty_value);

let repay_outcome = repay(vault, protocol, repay_amount)?;
protocol.realized_revenue_for_stakers =
    checked_add(protocol.realized_revenue_for_stakers, staker_penalty_nusd)?;
```

If the penalty is realized revenue, the direct addition should instead use the same target-aware router used by `repay()`:

```
protocol.route_realized_fee(liquidation_penalty_nusd)?;
```

Tool Used

Manual Review

Recommendation

First, confirm which single policy is authoritative for realized liquidation penalties: the liquidation-specific 60/40 keeper/insurance constants, or the target-aware `route_realized_fee()` policy. Do not apply both insurance percentages to the same penalty.

If the 60/40 policy is intended, apply it explicitly in both liquidation modes:

1. Compute the realized penalty after collateral caps, then allocate 60% to the keeper and 40% to insurance using the two constants and a documented rounding rule.
2. Return only principal plus the keeper's penalty share in `keeper_collateral_raw`, and return the insurance share in `insurance_collateral_raw`. Derive one side as the remainder so rounding cannot make the two outputs exceed the total seized collateral.
3. Do not also book the same insurance/keeper penalty value as `staker_penalty_nusd`; any separate staker charge should have its own explicitly documented basis to prevent double counting.
4. Preserve the same allocation when two-step liquidation collateral is settled instead of treating all proceeds above principal as generic PSM yield.
5. Add regression tests for the 30_000_000 repayment case, asserting a 1_440_000 keeper penalty and a 960_000 insurance penalty, plus tests for collateral exhaustion and raw-token rounding.

These changes should restore the invariant that the keeper and insurance allocations sum to the realized liquidation penalty and that the configured insurance collateral vault is actually funded by liquidation penalties.

If the 60/40 constants are obsolete and liquidation penalties are realized protocol revenue, remove the unused constants and replace the direct staker-revenue addition with:

```
let liquidation_penalty_nusd = core::cmp::min(penalty_value,
↳ delivered_penalty_value);

let repay_outcome = repay(vault, protocol, repay_amount)?;
protocol.route_realized_fee(liquidation_penalty_nusd)?;
```

The outcome should continue to expose the total penalty charged to the liquidator, while the resulting insurance and staker deltas determine the destination nUSD vault transfers. Add regression coverage for insurance below target, at target, and close enough to target that the insurance allocation is capped. For the default 30_000_000 repayment case, this policy interpretation should assert that the 2_400_000 penalty adds 480_000 to insurance and 1_920_000 to pre-targeted staker revenue.

Discussion

ducksquadd

<https://github.com/NestUSD/contracts/commit/668ea58ee54a530623df496d0091f5e807753725>

0502lian

Fixed

Issue M-8: Liquidation authority rotation can strand open two-step receipts [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/issues/69>

Summary

Open two-step liquidation receipts are bound to the liquidator that started them, but settlement also requires the signer to be the protocol's current `liquidation_authority`. If the admin rotates `liquidation_authority` while receipts are still open, the old authority fails the current-authority check and the new authority fails the receipt's `has_one = liquidator` constraint.

Vulnerability Detail

When a two-step liquidation is started, the receipt stores the current liquidator:

https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/ix/liquidation/two_step.rs#L41-44

```
ctx.accounts.liquidation_receipt.protocol = ctx.accounts.protocol.key();
ctx.accounts.liquidation_receipt.collateral_config =
    ↪ ctx.accounts.collateral_config.key();
ctx.accounts.liquidation_receipt.vault = ctx.accounts.vault.key();
ctx.accounts.liquidation_receipt.liquidator = ctx.accounts.liquidator.key();
```

The start instruction requires that liquidator to equal the current protocol liquidation authority:

https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/ix/liquidation/two_step.rs#L3-8

```
require_keys_eq!(
    ctx.accounts.protocol.liquidation_authority,
    ctx.accounts.liquidator.key(),
    CoreError::Unauthorized
);
```

Settlement later has two independent constraints. First, the account validation requires the receipt's stored liquidator to match the provided signer:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/accounts/liquidation.rs#L90-98>

```
#[account(
    mut,
    close = liquidator,
```

```

    has_one = protocol,
    has_one = liquidator,
    seeds = [b"liquidation", liquidation_receipt.vault.as_ref()],
    bump = liquidation_receipt.bump
  )]
  pub liquidation_receipt: Box<Account<'info, LiquidationReceipt>>,

```

Second, the settlement instruction again requires that signer to equal the current protocol liquidation authority:

https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/ix/liquidation/two_step.rs#L128-133

```

require_keys_eq!(
    ctx.accounts.protocol.liquidation_authority,
    ctx.accounts.liquidator.key(),
    CoreError::Unauthorized
);

```

The admin can rotate the liquidation authority at any time:

```

pub fn set_liquidation_authority(
    ctx: Context<MutateProtocol>,
    liquidation_authority: Pubkey,
) -> Result<()> {
    require_keys_neq!(
        liquidation_authority,
        Pubkey::default(),
        CoreError::InvalidParameter
    );
    ctx.accounts.protocol.liquidation_authority = liquidation_authority;
    Ok(())
}

```

This creates a stranded state:

```

Receipt was opened by authority A.
Admin rotates liquidation_authority to B.
A passes has_one = liquidator, but fails current-authority check.
B passes current-authority check, but fails has_one = liquidator.

```

Therefore, neither authority can settle the existing receipt unless the admin rotates the protocol authority back to A.

Impact

Any open two-step liquidation receipt can become unseizable after liquidation-authority rotation. This can leave seized collateral and pending liquidation accounting stuck until the admin restores the old authority.

Tool Used

Manual Review

Recommendation

Allow settlement by either the receipt's original liquidator or the current liquidation authority, or provide an admin migration path that can reassign open receipts during authority rotation.

Discussion

ducksquadd

<https://github.com/NestUSD/contracts/commit/9dda59a02a03c07ee3540ba2e46d9d6716be2022>

0502lian

Fixed

Issue M-9: Health factor flooring can incorrectly allow full liquidation [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/issues/71>

Summary

`health_factor_bps()` floors the collateral liquidation limit before calculating the health-factor ratio. `max_liquidation_repay()` compares this double-rounded result with `full_liquidation_threshold_bps`, so a vault whose exact health factor is slightly above the full-liquidation threshold can be reported as below it.

The configured liquidation authority can consequently initiate the two-step full-liquidation path for a vault that should only be partially liquidated. That path zeroes the vault and transfers all of its collateral to the liquidator; settlement then accounts for proceeds above principal as revenue rather than preserving the borrower's remaining equity.

Vulnerability Detail

The intended full-liquidation boundary is the exact comparison:

```
collateral_value_usd * liquidation_threshold_bps
< debt * full_liquidation_threshold_bps
```

Instead, `health_factor_bps()` first rounds the liquidation limit down to whole nUSD base units and then rounds the resulting ratio down to whole basis points:

```
let liq_limit = mul_div_down(
    collateral_value_usd,
    liquidation_threshold_bps as u128,
    BPS_DENOMINATOR,
)?;
mul_div_down(liq_limit, BPS_DENOMINATOR, debt)
```

`max_liquidation_repay()` uses that rounded result directly to select full liquidation:

```
let hf = health_factor_bps(collateral_value_usd, debt,
    ↪ params.liquidation_threshold_bps)?;
// ...
let hard_full = hf < params.full_liquidation_threshold_bps as u128;
// ...
let max_repay = if full { debt } else { close_factor_repay };
```

For example, use the following valid state:

collateral value	= 1,454,545,454,547 (\$1,454,545.454547)
debt	= 1,000,000,000,001 (\$1,000,000.000001)
liquidation threshold	= 5,500 bps
full-liquidation threshold	= 8,000 bps
close factor	= 5,000 bps
liquidation penalty	= 800 bps

Cross multiplication proves that the exact health factor is above the full-liquidation threshold:

```
1,454,545,454,547 * 5,500 > 1,000,000,000,001 * 8,000
exact HF = 8,000.0000000005 bps
```

However, the implementation calculates:

```
liq_limit = floor(1,454,545,454,547 * 5,500 / 10,000)
            = 800,000,000,000
HF         = floor(800,000,000,000 * 10,000 / 1,000,000,000,001)
            = 7,999 bps
```

This makes `hard_full` true and causes `max_liquidation_repay()` to return the entire debt with `full = true`. A correct close-factor liquidation would repay 500,000,000,001, apply a 40,000,000,001 penalty, and leave 914,545,454,545 of collateral value against 500,000,000,000 of debt. The remaining vault health factor would be 10,059 bps, demonstrating that full liquidation is unnecessary.

`start_liquidation_with_oracle()` relies on this Boolean result as its eligibility check:

```
let (_, full_liquidation) = domain::max_liquidation_repay(
    domain_vault(&ctx.accounts.vault),
    collateral_value,
    domain_params(&ctx.accounts.collateral_config, &ctx.accounts.protocol),
)
.map_err(map_core_error)?;
require!(full_liquidation, CoreError::InvalidParameter);
```

After the incorrect check passes, the instruction clears the vault and transfers all collateral:

```
ctx.accounts.vault.collateral_raw = 0;
ctx.accounts.vault.principal_debt = 0;
ctx.accounts.vault.accrued_fee = 0;
// ...
token_transfer_checked(
    // ...
    collateral_seized,
    // ...
```

```
)?;
```

Finally, `settle_liquidation_proceeds()` classifies every settlement unit above principal as revenue:

```
let revenue_amount = (usdc_amount as u128)
    .checked_sub(principal_debt)
    .ok_or(error!(CoreError::MathOverflow))?;
if revenue_amount > 0 {
    // route the amount through PSM yield accounting
}
```

The liquidation authority does not need to be malicious or misconfigured. A correctly configured authority following the normal liquidation flow can trigger the issue whenever ordinary price movement places a vault within the narrow rounding interval.

Impact

An affected borrower can be fully liquidated even though the exact protocol threshold permits only partial liquidation and that partial liquidation would restore the vault to health. In the demonstrated state, correct partial liquidation preserves 414,545,454,545 atomic units (\$414,545.454545) of borrower net equity, whereas the erroneous two-step full liquidation zeroes the vault and routes value above principal through protocol revenue accounting.

The loss is severe for each affected vault, although likelihood is reduced because the exploitable state is a sub-1-basis-point boundary and execution requires the configured liquidation authority. Given the direct loss of substantially more than 1% of the affected borrower's equity, High severity is appropriate under the supplied impact criteria.

Code Snippet

The following regression test reproduces the incorrect classification and shows that close-factor liquidation restores the vault's health:

```
use nest_domain::{
    health_factor_bps, max_liquidation_repay, mul_div_up, CollateralParams, Vault,
    BPS_DENOMINATOR,
};

#[test]
fn exact_hf_above_8000_is_misclassified_as_full_liquidation() {
    let collateral_value = 1_454_545_454_547_u128;
    let debt = 1_000_000_000_001_u128;
    let params = CollateralParams {
        liquidation_threshold_bps: 5_500,
        liquidation_penalty_bps: 800,
```



```

        close_factor_bps: 5_000,
        full_liquidation_threshold_bps: 8_000,
        ..CollateralParams::default()
    };
    let vault = Vault {
        collateral_raw: collateral_value,
        principal_debt: debt,
        accrued_fee: 0,
        last_accrual_ts: 0,
    };

    // The exact health factor is strictly above the 8,000 bps boundary.
    assert!(collateral_value * 5_500 > debt * 8_000);

    // Intermediate flooring makes the implementation report 7,999 bps.
    assert_eq!(
        health_factor_bps(collateral_value, debt, 5_500).unwrap(),
        7_999
    );
    let (max_repay, full) = max_liquidation_repay(vault, collateral_value,
        ↪ params).unwrap();
    assert_eq!(max_repay, debt);
    assert!(full);

    // A close-factor liquidation would restore the vault above 10,000 bps.
    let partial_repay = mul_div_up(debt, 5_000, BPS_DENOMINATOR).unwrap();
    let penalty = mul_div_up(partial_repay, 800, BPS_DENOMINATOR).unwrap();
    let remaining_collateral = collateral_value - partial_repay - penalty;
    let remaining_debt = debt - partial_repay;
    assert_eq!(partial_repay, 500_000_000_001);
    assert_eq!(penalty, 40_000_000_001);
    assert_eq!(remaining_collateral, 914_545_454_545);
    assert_eq!(remaining_debt, 500_000_000_000);
    assert_eq!(
        health_factor_bps(remaining_collateral, remaining_debt, 5_500).unwrap(),
        10_059
    );

    let two_step_revenue = collateral_value - debt;
    let borrower_equity_after_correct_partial = remaining_collateral -
        ↪ remaining_debt;
    assert_eq!(two_step_revenue, 454_545_454_546);
    assert_eq!(borrower_equity_after_correct_partial, 414_545_454_545);
}

```

Run from the repository root:

```
cd contracts
```

```
CARGO_TARGET_DIR=/tmp/nest-hf-poc-target cargo test -p nest-domain --test  
↳ hf_flooring_poc -- --nocapture
```

The test completes with:

```
test exact_hf_above_8000_is_misclassified_as_full_liquidation ... ok
```

The existing `nest-domain` test suite also passes all 44 tests and does not cover this boundary.

Tool Used

Manual Review

Recommendation

Do not use a health factor rounded to whole basis points to make the full-liquidation decision. Compare the exact products directly, using an overflow-safe wide-integer or full-precision `mul_div` implementation:

```
collateral_value_usd * liquidation_threshold_bps  
    < debt * full_liquidation_threshold_bps
```

Alternatively, implement an explicit comparison helper that preserves the rational value without intermediate division. Continue using a display-oriented rounded health factor only for reporting, not eligibility decisions. Add regression tests for values immediately below, exactly at, and immediately above both the full-liquidation and healthy-vault thresholds.

Discussion

ducksquadd

<https://github.com/NestUSD/contracts/commit/e44058a57e72d5282886007bba73f41ef71273ba>

0502lian

Fixed

Issue M-10: Lossy Kamino redemption leaves unrecovered principal recorded as deployed PSM backing [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/issues/72>

Summary

`redeem_psm_usdc_from_kamino` verifies that the requested Kamino cTokens were burned, but decreases `psm_kamino_deployed_usdc` only by the amount of USDC received. If a Kamino reserve has suffered a loss and its cToken exchange rate has fallen, a normal authority-initiated redemption can therefore destroy cTokens representing more recorded principal than the USDC returned. The difference remains recorded as deployed PSM backing even though the corresponding cTokens no longer exist.

The existing PSM invariant cannot detect this state because it compares `psm_idle_usdc + psm_kamino_deployed_usdc` only with liabilities and verifies only the idle balance against the USDC vault. The stale deployed value is also included in the staking program's revenue-capacity calculation. If the protocol remains unpaused, public PSM users can consume the recovered idle USDC at par while the external loss remains hidden in accounting, leaving later holders exposed to the shortfall.

Vulnerability Detail

The `PsmKamino` account context restricts redemption to `protocol.authority`, so this issue does not require a malicious public caller or an authority misconfiguration. It occurs when the trusted authority performs the intended recovery operation after an external Kamino principal loss.

In `contracts/programs/nest-core/src/ix/kamino.rs`, `redeem_psm_usdc_from_kamino` first refreshes the reserve, redeems `collateral_amount`, and confirms that exactly that many cTokens left the protocol's collateral vault:

```
invoke_klend_refresh_reserve(&psm_kamino_cpi_accounts(ctx.accounts))?;
invoke_klend_redeem_reserve_collateral(
    &psm_kamino_cpi_accounts(ctx.accounts),
    collateral_amount,
    signer_seeds,
)?;

ctx.accounts.psm_usdc_vault.reload()?;
ctx.accounts.protocol_kamino_collateral_vault.reload()?;
require_token_account_decrease(
    kamino_collateral_before,
    ctx.accounts.protocol_kamino_collateral_vault.amount,
    collateral_amount,
```

```

)?;
let usdc_received = ctx
    .accounts
    .psm_usdc_vault
    .amount
    .checked_sub(psm_vault_before)
    .ok_or(error!(CoreError::TransferFeeNotSupported))?;
require!(usdc_received > 0, CoreError::TransferFeeNotSupported);

```

However, the function does not determine how much recorded principal was represented by the burned cTokens. Instead, it treats all received USDC as returned principal up to the entire deployed balance and subtracts only that receipt from `psm_kamino_deployed_usdc`:

```

let principal_return = core::cmp::min(
    usdc_received as u128,
    ctx.accounts.protocol.psm_kamino_deployed_usdc,
);

ctx.accounts.protocol.psm_kamino_deployed_usdc = ctx
    .accounts
    .protocol
    .psm_kamino_deployed_usdc
    .checked_sub(principal_return)
    .ok_or(error!(CoreError::MathOverflow))?;
ctx.accounts.protocol.psm_idle_usdc = ctx
    .accounts
    .protocol
    .psm_idle_usdc
    .checked_add(principal_return)
    .ok_or(error!(CoreError::MathOverflow))?;

```

To carry the state transition through, suppose the PSM records 100 USDC of Kamino principal and owns 100 cTokens. After a reserve loss, we redeem all 100 cTokens but receive only 80 USDC. The cToken decrease check passes, `principal_return` is 80, idle liquidity becomes 80, and deployed principal becomes 20. We now have no cTokens supporting that final 20.

The post-operation check in `contracts/programs/nest-core/src/helpers/psm.rs` does not compare the deployed value with the remaining cToken balance or exchange rate:

```

let tracked_psm_assets = protocol
    .psm_idle_usdc
    .checked_add(protocol.psm_kamino_deployed_usdc)
    .ok_or(error!(CoreError::MathOverflow))?;
require!(
    tracked_psm_assets <= protocol.psm_usdc_liabilities,
    CoreError::InvalidParameter
);

```

```
require_recorded_amount_covered(actual_psm_usdc, protocol.psm_idle_usdc)
```

Consequently, 80 idle + 20 deployed $\leq$ 100 liabilities passes, and the actual USDC vault covers the recorded idle 80. As we move through both checks, neither condition proves that the remaining deployed 20 exists.

The yield-only path shows that a principal-coverage proof is available but missing from ordinary redemption. `realize_psm_kamino_yield` calls `verify_kamino_profit_slice`, which uses the observed redemption rate to require that the remaining cTokens cover all tracked principal. Ordinary redemption does not call this helper and also permits a full cToken redemption, for which there is no remaining collateral to test.

The stale value then crosses a program boundary. In `contracts/programs/nest-stake/src/helpers.rs`, staking capacity treats both idle USDC and recorded Kamino principal as deployable assets:

```
let deployable_psm_usdc = psm_idle_usdc
    .checked_add(psm_kamino_deployed_usdc)
    .ok_or(error!(StakeError::MathOverflow))?;
let kamino_revenue = deployable_psm_usdc
    .checked_mul(kamino_apr_bps as u128)
    .ok_or(error!(StakeError::MathOverflow))?
    .checked_div(domain::BPS_DENOMINATOR)
    .ok_or(error!(StakeError::MathOverflow));
```

Thus, when we follow the stale value across the program boundary, the nonexistent 20 continues to contribute to anticipated Kamino revenue and staking capacity.

Separately, `psm_swap_out` is public and pays any amount covered by `psm_idle_usdc`; if the protocol is not paused, early users can redeem the recovered 80 at par. The external Kamino loss itself, and the fact that public exits can consume available idle liquidity, are not created by this accounting bug. The defect is that redemption fails to recognize the loss, allows invariants to report a fully tracked 100 of assets, and provides no automatic signal or pause before that liquidity is consumed.

Impact

Every discounted redemption can leave the difference between the principal represented by burned cTokens and the USDC received recorded as a nonexistent asset. This hides a real PSM shortfall from the protocol's accounting checks and overstates the asset base used to calculate staking revenue capacity.

In the demonstrated full-redemption case, the PSM has 100 of liabilities, receives only 80 USDC, and owns no cTokens, yet records 80 idle plus 20 deployed. If public users redeem the available 80 while the system remains unpaused, the final state is 20 of nUSD liabilities, zero idle USDC, zero cTokens, and 20 of stale deployed principal. Later holders then bear an unredeemable shortfall up to the actual Kamino loss.

The accounting failure is deterministic after a discounted redemption, but its likelihood is

limited by the prerequisites: Kamino must suffer a principal loss, the trusted protocol authority must redeem against the impaired exchange rate, and public first-exit effects require the protocol to remain unpaused. A vigilant authority can inspect the reserve and pause PSM outflows before redemption. Given the potentially material solvency impact but low-probability external-loss trigger, Medium severity is appropriate.

Code Snippet

The following standalone PoC models the exact relevant accounting operations and invariant checks:

```
fn main() {
    let mut liabilities: u128 = 100;
    let mut nusd_supply: u128 = 100;
    let mut idle: u128 = 0;
    let mut deployed: u128 = 100;
    let ctoken_before: u64 = 100;
    let collateral_amount: u64 = 100;
    let ctoken_after = ctoken_before.checked_sub(collateral_amount).unwrap();
    let usdc_received: u64 = 80;

    // Exact accounting performed by redeem_psm_usdc_from_kamino.
    let principal_return = core::cmp::min(usdc_received as u128, deployed);
    let redeemed_surplus = (usdc_received as u128)
        .checked_sub(principal_return)
        .unwrap();
    assert_eq!(principal_return + redeemed_surplus, usdc_received as u128);
    deployed = deployed.checked_sub(principal_return).unwrap();
    idle = idle.checked_add(principal_return).unwrap();

    // Exact logical checks in require_psm_accounting_invariants.
    assert_eq!(liabilities, nusd_supply);
    assert!(idle + deployed <= liabilities);
    assert!(usdc_received as u128 >= idle);

    // All cTokens are gone, but 20 USDC remains recorded as deployed.
    assert_eq!(ctoken_after, 0);
    assert_eq!((idle, deployed), (80, 20));

    // Exact state effects of a public psm_swap_out for the recovered 80 USDC.
    let amount: u128 = 80;
    assert!(idle >= amount);
    liabilities = liabilities.checked_sub(amount).unwrap();
    nusd_supply = nusd_supply.checked_sub(amount).unwrap();
    idle = idle.checked_sub(amount).unwrap();

    assert_eq!((liabilities, nusd_supply, idle, deployed), (20, 20, 0, 20));
    assert_eq!(ctoken_after, 0);
    println!(
```

```
        "liabilities={liabilities} idle={idle} deployed={deployed}  
        ↪  ctokens={ctoken_after}"  
    );  
}
```

Save the code as `poc_lossy_redeem.rs`, then run:

```
rustc poc_lossy_redeem.rs -o /tmp/poc_lossy_redeem  
/tmp/poc_lossy_redeem
```

Expected output:

```
liabilities=20 idle=0 deployed=20 ctokens=0
```

Tool Used

Manual Review

Recommendation

Account for the principal represented by the cTokens being burned, independently of the USDC actually returned. For a full redemption, all recorded Kamino principal must be removed from `psm_kamino_deployed_usdc`; only the USDC actually received should be added to `psm_idle_usdc`, and any difference must be explicitly recognized as a loss. For partial redemptions, allocate principal to the redeemed cTokens using a conservative, documented rounding rule and prove that the remaining cToken value covers the remaining recorded principal.

Reuse or generalize the coverage logic behind `verify_kamino_profit_slice` so every redemption enforces the relationship between remaining cTokens and remaining deployed principal. Because a full redemption has no remaining slice to prove, handle it explicitly by requiring the deployed balance to become zero.

If a redemption realizes any principal shortfall, automatically pause PSM outflows or enter an explicit loss-handling mode before recovered idle liquidity becomes publicly redeemable. Staking capacity should consume only verified, loss-adjusted PSM assets rather than an unproven book balance.

Discussion

ducksquadd

<https://github.com/NestUSD/contracts/commit/d0cad5bf222b102f723a5acf42f74139f277ecc3>

0502lian

Issue L-1: Unauthenticated fixed-PDA initializers allow first caller to seize protocol and staking authority [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/issues/57>

Summary

`initialize_protocol` and `initialize_staking` create singleton PDAs from fixed seeds, but they accept any signer as the initial authority. If either singleton PDA is still uninitialized, an attacker can initialize it first with compliant mint and vault accounts, permanently becoming the protocol or staking authority because Anchor `init` prevents the legitimate deployer from reinitializing the same PDA.

Vulnerability Detail

The protocol state is a singleton PDA derived only from the fixed seed `b"protocol"`. The initializer payer and future authority is an arbitrary signer; there is no constraint that it matches a known deployer, governance address, or preconfigured admin:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/accounts/setup.rs#L1-L23>

```
#[derive(Accounts)]
pub struct InitializeProtocol<'info> {
    #[account(init, payer = authority, space = 8 + Protocol::INIT_SPACE, seeds =
        ↪ [b"protocol"], bump)]
    pub protocol: Box<Account<'info, Protocol>>,
    pub nusd_mint: InterfaceAccount<'info, Mint>,
    pub usdc_mint: InterfaceAccount<'info, Mint>,
    #[account(mut, token::mint = usdc_mint, token::authority = protocol,
        ↪ token::token_program = usdc_token_program)]
    pub psm_usdc_vault: InterfaceAccount<'info, TokenAccount>,
    ...
    #[account(mut)]
    pub authority: Signer<'info>,
```

The initialization logic performs shape checks on the provided mint and vault accounts, but then directly stores the caller as both the protocol admin and liquidation authority:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/ix/setup.rs#L54-L88>

```
require!(
    ctx.accounts.nusd_mint.mint_authority ==
        ↪ COption::Some(ctx.accounts.protocol.key()),
```

```

        CoreError::InvalidParameter
    );
    ...
    let protocol = &mut ctx.accounts.protocol;
    protocol.authority = ctx.accounts.authority.key();
    protocol.liquidation_authority = ctx.accounts.authority.key();
    protocol.nusd_mint = ctx.accounts.nusd_mint.key();
    protocol.usdc_mint = ctx.accounts.usdc_mint.key();
    protocol.psm_usdc_vault = ctx.accounts.psm_usdc_vault.key();

```

These shape checks do not authenticate the deployer. An attacker can create an empty nUSD mint whose mint authority is the future protocol PDA, create empty token accounts owned by the protocol PDA, and call `initialize_protocol` first. SPL Token account and mint creation can assign PDA authorities without that PDA signing, so the attacker can satisfy the checks before the protocol PDA account exists.

After initialization, privileged protocol instructions are gated by the authority value that the attacker wrote during initialization:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/accounts/admin.rs#L1-L22>

```

#[derive(Accounts)]
pub struct MutateProtocol<'info> {
    #[account(mut, seeds = [b"protocol"], bump = protocol.bump, has_one =
        ↪ authority)]
    pub protocol: Box<Account<'info, Protocol>>,
    pub authority: Signer<'info>,
}

#[derive(Accounts)]
pub struct SetNestPriceSigner<'info> {
    #[account(seeds = [b"protocol"], bump = protocol.bump, has_one = authority)]
    pub protocol: Box<Account<'info, Protocol>>,
    ...
    pub authority: Signer<'info>,
}

```

That attacker-controlled authority can pause the protocol, transfer authority, set the liquidation authority, and configure the signed oracle signer:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/ix/admin.rs#L3-L70>

```

pub fn set_paused(ctx: Context<MutateProtocol>, paused: bool) -> Result<()> {
    ctx.accounts.protocol.paused = paused;
    Ok(())
}
...
pub fn set_protocol_authority(ctx: Context<MutateProtocol>, authority: Pubkey) ->
    ↪ Result<()> {

```

```

    require_keys_neq!(authority, Pubkey::default(), CoreError::InvalidParameter);
    ctx.accounts.protocol.authority = authority;
    Ok(())
}

```

```

pub fn set_nest_price_signer(ctx: Context<SetNestPriceSigner>, signer: Pubkey) ->
↳ Result<()> {
    require_keys_neq!(signer, Pubkey::default(), CoreError::InvalidParameter);
    let config = &mut ctx.accounts.nest_price_signer;
    config.protocol = ctx.accounts.protocol.key();
    config.authority = ctx.accounts.authority.key();
    config.signer = signer;
}

```

The same pattern exists in the staking program. The staking state is a singleton PDA derived only from b"staking", and the initializer accepts any signer:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-stake/src/accounts.rs#L1-L18>

```

#[derive(Accounts)]
pub struct InitializeStaking<'info> {
    #[account(init, payer = authority, space = 8 + StakingState::INIT_SPACE, seeds
↳ = [b"staking"], bump)]
    pub staking_state: Box<Account<'info, StakingState>>,
    pub nusd_mint: InterfaceAccount<'info, Mint>,
    pub snusd_mint: InterfaceAccount<'info, Mint>,
    ...
    #[account(mut)]
    pub authority: Signer<'info>,
}

```

The caller is then stored as the staking authority:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-stake/src/instructions.rs#L39-L58>

```

let state = &mut ctx.accounts.staking_state;
state.authority = ctx.accounts.authority.key();
state.nusd_mint = ctx.accounts.nusd_mint.key();
state.snusd_mint = ctx.accounts.snusd_mint.key();
state.staking_nusd_vault = ctx.accounts.staking_nusd_vault.key();
state.revenue_nusd_vault = ctx.accounts.revenue_nusd_vault.key();
...
state.paused = false;
state.bump = ctx.bumps.staking_state;
}

```

Subsequent staking admin and loss-realization instructions rely on this stored authority:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-stake/src/accounts.rs#L20-L38>

```

#[derive(Accounts)]
pub struct MutateStake<'info> {
    #[account(mut, seeds = [b"staking"], bump = staking_state.bump, has_one =
        ↪ authority)]
    pub staking_state: Box<Account<'info, StakingState>>,
    pub authority: Signer<'info>,
}

#[derive(Accounts)]
pub struct RealizeLoss<'info> {
    #[account(mut, seeds = [b"staking"], bump = staking_state.bump, has_one =
        ↪ authority)]
    pub staking_state: Box<Account<'info, StakingState>>,

```

A concrete attack flow is:

1. Before the legitimate deployer initializes the singleton protocol PDA, the attacker creates compliant empty mints and token accounts. The nUSD mint authority is set to the future protocol PDA, and the required vaults are owned by that PDA.
2. The attacker calls `initialize_protocol` with themselves as authority.
3. The fixed `[b"protocol"]` PDA is now initialized, so the legitimate deployer cannot initialize the real protocol state.
4. The attacker uses protocol authority to call admin setters such as `set_nest_price_signer`, `add_collateral`, `set_paused`, risk setters, and authority transfer functions.
5. With attacker-controlled signed prices and attacker-added collateral, the attacker can mint nUSD through the protocol's configured mint path. If real PSM USDC liquidity is later deposited into the attacker-initialized protocol vault, the attacker can burn the minted nUSD through `psm_swap_out` and withdraw USDC.
6. Separately, if the staking singleton is uninitialized, the attacker can initialize `[b"staking"]` first and then control staking pause, staking authority transfer, and authority-only loss realization.

Impact

An attacker can permanently seize the protocol or staking singleton authority during deployment if the corresponding PDA is uninitialized. This blocks the legitimate deployment and gives the attacker full control over admin-gated functionality, including pausing the protocol, transferring authorities, configuring oracle signers, adding collateral markets, changing risk parameters, and controlling staking administration.

If users or deployment scripts later interact with the attacker-initialized state and deposit real PSM liquidity, the issue can escalate from governance and deployment takeover to loss of USDC liquidity. The attacker can configure the system to mint nUSD

against attacker-controlled collateral and redeem that nUSD against later PSM USDC liquidity.

Code Snippet

The vulnerable initializers are both fixed-seed init accounts whose authority is not authenticated:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/accounts/setup.rs#L1-L18>

```
#[account(init, payer = authority, space = 8 + Protocol::INIT_SPACE, seeds =
↳ [b"protocol"], bump)]
pub protocol: Box<Account<'info, Protocol>>,
...
pub authority: Signer<'info>,
```

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-stake/src/accounts.rs#L1-L12>

```
#[account(init, payer = authority, space = 8 + StakingState::INIT_SPACE, seeds =
↳ [b"staking"], bump)]
pub staking_state: Box<Account<'info, StakingState>>,
...
pub authority: Signer<'info>,
```

Both instructions persist that arbitrary signer as the future admin:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/ix/setup.rs#L79-L81>

```
let protocol = &mut ctx.accounts.protocol;
protocol.authority = ctx.accounts.authority.key();
protocol.liquidation_authority = ctx.accounts.authority.key();
```

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-stake/src/instructions.rs#L39-L40>

```
let state = &mut ctx.accounts.staking_state;
state.authority = ctx.accounts.authority.key();
```

Tool Used

Manual Review

Recommendation

Authenticate singleton initialization. For example, require the initializing authority to equal a known deployer, governance, or upgrade authority address:

```
[account(mut, address = EXPECTED_INITIAL_AUTHORITY)]  
pub authority: Signer<'info>,
```

Also bind initialization to canonical deployment assets. `initialize_protocol` should constrain the expected USDC mint, nUSD mint, and protocol-owned vaults rather than accepting arbitrary compliant accounts. `initialize_staking` should bind to the canonical protocol state and canonical nUSD mint, not only to the derived protocol PDA as mint authority.

Discussion

ducksquadd

<https://github.com/NestUSD/contracts/commit/f395e5d7790fc9b0015f95315ae54c41df7bff4e>

0502lian

Fixed

Issue L-2: Rounded-zero fee accrual leaves stale timestamps and can overcharge later borrows [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/issues/61>

Summary

`accrue_stability_fee` advances `vault.last_accrual_ts` when the vault has no debt or when the configured stability fee APR is zero. However, it does not advance the timestamp when the calculated fee rounds down to zero.

This lets a vault keep a stale accrual timestamp while carrying dust debt. If the borrower later mints a large amount, the next stability fee accrual charges the newly borrowed amount for the entire stale interval, even though that larger debt did not exist during that time.

Vulnerability Detail

The stability fee path computes fees from the current vault debt and the elapsed time since `last_accrual_ts`:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/crates/nest-domain/src/cdp.rs#L161-L181>

```
let elapsed = now
    .checked_sub(vault.last_accrual_ts)
    .ok_or(NestError::MathOverflow)? as u128;
let debt = vault.total_debt()?;
if debt == 0 {
    vault.last_accrual_ts = now;
    return Ok(0);
}
if stability_fee_apr_bps == 0 {
    vault.last_accrual_ts = now;
    return Ok(0);
}
let numerator = debt
    .checked_mul(stability_fee_apr_bps as u128)
    .and_then(|v| v.checked_mul(elapsed))
    .ok_or(NestError::MathOverflow)?;
let denominator = BPS_DENOMINATOR
    .checked_mul(SECONDS_PER_YEAR)
    .ok_or(NestError::MathOverflow)?;
let fee = numerator
    .checked_div(denominator)
```

The division is integer division. When the exact fee is smaller than one atomic unit, fee becomes zero. In that case, the function returns without updating `vault.last_accrual_ts`:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/crates/nest-domain/src/cdp.rs#L180-L189>

```
let fee = numerator
    .checked_div(denominator)
    .ok_or(NestError::DivisionByZero)?;
if fee == 0 {
    return Ok(0);
}
vault.accrued_fee = checked_add(vault.accrued_fee, fee)?;
protocol.total_debt = checked_add(protocol.total_debt, fee)?;
protocol.total_uncollected_fees = checked_add(protocol.total_uncollected_fees,
    ↪ fee)?;
vault.last_accrual_ts = now;
```

The existing domain test already asserts that rounded-zero accrual leaves the timestamp stale:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/crates/nest-domain/src/tests.rs#L260-L275>

```
let too_early =
    accrue_stability_fee(&mut vault, &mut protocol, DEFAULT_STABILITY_FEE_BPS,
    ↪ 1).unwrap();
assert_eq!(too_early, 0);
assert_eq!(vault.last_accrual_ts, 0);
assert_eq!(vault.accrued_fee, 0);
assert_eq!(protocol.total_uncollected_fees, 0);
```

This would be mostly harmless if the vault's debt could not change while the timestamp is stale. However, borrowing first accrues the vault fee, then increases principal:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/ix/cdp.rs#L143-L151>

```
pub fn mint_nusd_with_oracle(ctx: Context<MintNusdWithOracle>, amount: u64) ->
    ↪ Result<> {
    require!(amount > 0, CoreError::InvalidParameter);
    require!(!ctx.accounts.protocol.paused, CoreError::Paused);
    accrue_vault_fee(
        &mut ctx.accounts.vault,
        &mut ctx.accounts.protocol,
        &mut ctx.accounts.collateral_config,
        Clock::get()?.unix_timestamp,
    )?;
```


The domain borrow logic then adds the new principal without changing `last_accrual_ts`:
<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/crates/nest-domain/src/cdp.rs#L245-L261>

```
pub fn borrow(
    vault: &mut Vault,
    protocol: &mut ProtocolAccounting,
    collateral_value_usd: u128,
    params: CollateralParams,
    borrow_amount: u128,
) -> Result<()> {
    can_borrow(
        collateral_value_usd,
        *vault,
        params,
        *protocol,
        borrow_amount,
    )?;
    vault.principal_debt = checked_add(vault.principal_debt, borrow_amount)?;
    protocol.total_debt = checked_add(protocol.total_debt, borrow_amount)?;
    Ok(())
}
```

As a result, if the pre-borrow accrual rounded down to zero, the stale timestamp remains attached to the vault after the principal has become much larger.

A concrete example using the default 150 bps stability fee APR and 6-decimal nUSD:

1. A vault has `principal_debt = 1` atomic unit and `last_accrual_ts = t0`.
2. After 30 days, the borrower mints 100,000 nUSD.
3. The pre-borrow accrual computes:

```
floor(1 * 150 * 2,592,000 / (10,000 * 31,536,000)) = 0
```

4. Because `fee == 0`, `last_accrual_ts` remains `t0`.
5. The borrow increases principal to 100,000,000,001 atomic units.
6. One second later, the next accrual computes against the large debt for the full stale interval:

```
floor(100,000,000,001 * 150 * 2,592,001 / (10,000 * 31,536,000))
= 123,287,718 atomic units
= 123.287718 nUSD
```

The correct time-weighted fee for the same scenario would be only the zero-rounded dust-debt fee for the first 30 days plus about one second of fee on the large borrow:

```
floor(100,000,000,001 * 150 * 1 / (10,000 * 31,536,000))  
= 47 atomic units  
= 0.000047 nUSD
```

The excess fee is created because the stale dust-debt interval is retroactively applied to the newly borrowed principal.

Impact

Affected borrowers can be charged unearned stability fees. The overcharge increases `vault.accrued_fee`, `protocol.total_debt`, and collateral-level debt accounting. If the vault is close to its borrow limit or liquidation threshold, the inflated debt can make the vault liquidatable and put collateral at risk.

The stale dust state is owner-created, so this is not arbitrary third-party theft against unrelated vaults. However, once the debt is inflated, normal liquidation flows can act on the inflated accounting state.

Code Snippet

The vulnerable early return skips the timestamp update after rounded-zero accrual:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/crates/nest-domain/src/cdp.rs#L180-L189>

Borrowing calls accrual before increasing principal:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/ix/cdp.rs#L146-L169>

Tool Used

Manual Review

Recommendation

For any positive stability-fee accrual numerator, charge at least 1 atomic unit and advance `vault.last_accrual_ts`. This prevents both stale-timestamp overcharging and repeated rounded-zero fee avoidance.

Because nUSD is constrained to `STABLECOIN_DECIMALS = 6`, the minimum rounded-up fee is only 0.000001 nUSD. This is a small amount, but it preserves the invariant that any positive elapsed debt interval at a positive APR eventually produces a fee and cannot be repeatedly truncated to zero by frequent accrual calls.

For example:

```

let mut fee = numerator
    .checked_div(denominator)
    .ok_or(NestError::DivisionByZero)?;

if fee == 0 && numerator > 0 {
    fee = 1;
}

vault.last_accrual_ts = now;
if fee == 0 {
    return Ok(0);
}

```

Then apply the normal fee accounting for the possibly rounded-up `fee`. The existing rounded-zero test should also be updated to assert the new minimum-fee behavior rather than expecting the timestamp to remain stale.

Discussion

ducksquadd

<https://github.com/NestUSD/contracts/commit/c94dfc2d369a1b34a7d0173c0fb446fa3f8ca072>

0502lian

Fixed

Issue L-3: Closed-market oracle risk controls are configured but not applied [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/issues/66>

Summary

Collateral configuration includes closed-market, underlying-price, and redemption-rate risk parameters, but the actual pricing path only uses the xStock/USD oracle feed. As a result, configured protections such as closed-market haircuts, underlying-feed staleness checks, and redemption-rate sanity checks are not enforced when valuing collateral.

Vulnerability Detail

CollateralConfig stores multiple oracle and closed-market risk fields:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/state.rs#L62-73>

```
pub xstock_usd_feed_id: [u8; 32],
pub underlying_usd_feed_id: [u8; 32],
pub redemption_rate_feed_id: [u8; 32],
pub max_staleness_seconds: i64,
pub closed_market_max_staleness_seconds: i64,
pub underlying_closed_market_max_staleness_seconds: i64,
pub closed_market_haircut_bps: u16,
```

These parameters are validated and stored during collateral onboarding:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/ix/setup.rs#L150-157>

```
&& params.closed_market_max_staleness_seconds >= params.max_staleness_seconds
&& params.closed_market_max_staleness_seconds <= CLOSED_MARKET_MAX_STALENESS_SECONDS
&& params.underlying_closed_market_max_staleness_seconds
    >= params.closed_market_max_staleness_seconds
&& params.underlying_closed_market_max_staleness_seconds
    <= UNDERLYING_CLOSED_MARKET_MAX_STALENESS_SECONDS
&& params.closed_market_haircut_bps > 0
&& params.closed_market_haircut_bps <= MAX_CLOSED_MARKET_HAIRCUT_BPS
```

However, the actual domain pricing function ignores the underlying feed, redemption-rate feed, market state, and closed-market haircut:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/crates/nest-domain/src/oracle.rs#L76-80>

```
pub fn safe_raw_token_price_e8(inputs: PricingInputs) -> Result<u128> {
    // Collateral is priced from the xStock/USD feed. The remaining feed fields
    // are inert under this policy but still validated by the shared input type.
    lower_confidence_bound(inputs.xstock_usd, inputs.xstock_policy, inputs.now)
}
```

The oracle helper passes all three feeds into PricingInputs, but only xstock_usd affects the returned price:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/helpers/oracle.rs#L55-64>

```
domain::safe_raw_token_price_e8(domain::PricingInputs {
    xstock_usd: to_domain_price(oracle.xstock_usd),
    underlying_usd: to_domain_price(oracle.underlying_usd),
    redemption_rate: to_domain_price(oracle.redemption_rate),
    now: clock.unix_timestamp,
    xstock_policy: policy(config.xstock_usd_feed_id, config)?,
    underlying_policy: policy(config.underlying_usd_feed_id, config)?,
    redemption_policy: policy(config.redemption_rate_feed_id, config)?,
})
```

Additionally, oracle refresh stores inactive placeholder snapshots for the underlying and redemption feeds, and resets market state to regular:

<https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/blob/main/contracts/programs/nest-core/src/ix/oracle.rs#L39-44>

```
oracle.underlying_usd =
    inactive_oracle_price(ctx.accounts.collateral_config.underlying_usd_feed_id);
oracle.redemption_rate =
    inactive_oracle_price(ctx.accounts.collateral_config.redemption_rate_feed_id);
oracle.market_state = MarketStateAccount::Regular;
oracle.calendar_valid_until_ts = 0;
```

Therefore, the configured closed-market and secondary-feed risk controls are effectively absent.

Impact

If the intended risk model requires closed-market haircuts or underlying/redemption sanity checks, collateral can be overvalued during market closures. This is especially relevant for tokenized stock collateral, where market hours and redemption liquidity can materially affect collateral value.

Tool Used

Manual Review

Recommendation

Either implement the configured risk controls or remove the unused configuration to avoid false safety assumptions.

Discussion

ducksquadd

These params are just there in the event collateral needs some calendar things. At the moment all supported tokens trade 24/7.

ducksquadd

<https://github.com/NestUSD/contracts/commit/9374c093b6008fbbelc29c34eb45cdc b75cd55cc>

0502lian

Fixed

Issue L-4: Token-2022 mint capabilities are not validated during collateral onboarding [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/issues/73>

Summary

`add_collateral()` accepts Token-2022 mints but does not inspect their mint extensions or authorities. A listed collateral can therefore retain capabilities such as a permanent delegate, freeze authority or transfer hook.

Vulnerability Detail

`add_collateral()` validates basic parameters and token accounts but never parses Token-2022 mint extensions. Token-2022 is explicitly accepted by `valid_collateral_token_program()`:

```
pub(crate) fn valid_collateral_token_program(token_program: Pubkey) -> bool {
    token_program == SPL_TOKEN_PROGRAM_ID || token_program == TOKEN_2022_PROGRAM_ID
}
```

The only additional vault check is `require_token_account_unencumbered()`:

```
fn require_token_account_unencumbered(account: &InterfaceAccount<TokenAccount>) ->
    Result<()> {
    require!(
        account.delegate == COption::None
        && account.delegated_amount == 0
        && account.close_authority == COption::None
        && account.is_native == COption::None,
        CoreError::InvalidParameter
    );
    Ok(())
}
```

This only checks token-account fields. It does not detect mint-level capabilities. For example, a permanent delegate can transfer or burn tokens from the protocol collateral vault without calling NestUSD. The protocol's `total_deposits_raw`, borrower collateral, and debt would remain unchanged.

Impact

An external mint authority could remove or freeze deposited collateral, leaving nUSD underbacked and causing withdrawals or liquidations to fail.

Tool Used

Manual Review

Recommendation

Parse Token-2022 mint data during onboarding and enforce an extension allowlist.

Discussion

ducksquadd

<https://github.com/NestUSD/contracts/commit/b284db97fa402e964117d8508cbdc2714b6bc954>

ducksquadd

This is an inherent thing with any tokenized stock. Have added a guardrail though to stop mints when things are frozen and what not.

Issue L-5: Mainnet signing script imports @solana/spl-token from an untracked sibling project [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-07-nestusd-july-6th-2026/issues/74>

Summary

The mainnet setup script loads @solana/spl-token relative to ../v2/package.json rather than the audited contracts project. The dependency executed by the script is therefore not pinned by this repository, even though the same process loads the mainnet deployer key and signs privileged transactions.

Vulnerability Detail

The script creates a resolver rooted at an external sibling project and uses it to import @solana/spl-token:

```
import fs from "node:fs";
import path from "node:path";
import process from "node:process";
import { createRequire } from "node:module";

const contractsRoot = process.cwd();
const repoRoot = path.resolve(contractsRoot, "..");
const requireFromContracts = createRequire(path.join(contractsRoot,
  ↪ "package.json"));
const requireFromV2 = createRequire(path.join(repoRoot, "v2", "package.json"));

const anchor = requireFromContracts("@coral-xyz/anchor");
const { Connection, Keypair, PublicKey, SystemProgram } =
  ↪ requireFromContracts("@solana/web3.js");
const { createAccount, getMint, TOKEN_2022_PROGRAM_ID } =
  ↪ requireFromV2("@solana/spl-token");

const deploymentPath = path.join(repoRoot, "deployments", "mainnet-v1.json");
const idlPath = path.join(contractsRoot, "target", "idl", "nest_core.json");
const keypairPath = path.join(repoRoot, "mainnet-secrets",
  ↪ "nest-mainnet-deployer.json");
const artifactPath = path.join(repoRoot, "deployments",
  ↪ "mainnet-backpack-signed-stocks-2026-06-29.json");
```

The audited contracts/package.json does not declare @solana/spl-token:

```
{
  "name": "@nest-labs/contracts",
  "version": "0.1.0",
```

```

"private": true,
"type": "module",
"scripts": {
  "test": "npm run test:domain",
  "test:domain": "cargo test -p nest-domain",
  "check:programs": "cargo check -p nest-core && cargo check -p nest-stake",
  "test:workspace": "cargo test --workspace",
  "build:anchor": "anchor build",
  "test:anchor": "anchor test --provider.cluster localnet",
  "setup:backpack-stocks": "node scripts/setup-backpack-signed-stocks-mainnet.mjs"
},
"devDependencies": {
  "@coral-xyz/anchor": "0.31.1",
  "@solana/web3.js": "^1.98.0",
  "@types/node": "^24.0.0",
  "typescript": "^5.8.3"
}
}

```

The script reads the deployer secret key through `keypairFromJson()`:

```

function readJson(filePath) {
  return JSON.parse(fs.readFileSync(filePath, "utf8"));
}

function writeJson(filePath, value) {
  fs.writeFileSync(filePath, `${JSON.stringify(value, null, 2)}\n`);
}

function keypairFromJson(filePath) {
  return Keypair.fromSecretKey(Uint8Array.from(readJson(filePath)));
}

```

The key is then loaded into the same process that already executed the external dependency:

```

const deployment = readJson(deploymentPath);
const idl = readJson(idlPath);
requireInstruction(idl, "set_nest_price_signer");
requireInstruction(idl, "add_collateral");

const payer = keypairFromJson(keypairPath);
const connection = new Connection(
  process.env.NEST_MAINNET_RPC
  ?? process.env.NEXT_PUBLIC_NEST_RPC_ENDPOINT
  ?? deployment.rpcEndpoint
  ?? "https://api.mainnet-beta.solana.com",
  "confirmed",
);

```

```
const provider = new anchor.AnchorProvider(connection, new anchor.Wallet(payer), {
  commitment: "confirmed",
  preflightCommitment: "confirmed",
});
const program = new anchor.Program(idl, provider);
```

The payer signs privileged transactions:

```
if (!skipSigner) {
  artifact.priceSigner = priceSigner.toBase58();
  artifact.nestPriceSigner = nestPriceSigner.toBase58();
  if (execute) {
    console.log("setting on-chain price signer");
    const signature = await program.methods
      .setNestPriceSigner(priceSigner)
      .accounts({
        protocol,
        nestPriceSigner,
        authority: payer.publicKey,
        systemProgram: SystemProgram.programId,
      })
      .signers([payer])
      .rpc();
    artifact.signatures.setNestPriceSigner = signature;
    writeJson(artifactPath, artifact);
    console.log("set_nest_price_signer confirmed", signature);
  }
}
```

Impact

A compromised sibling dependency can expose the mainnet deployer key or alter privileged deployment transactions.

Tool Used

Manual Review

Recommendation

Declare an exact @solana/spl-token version in contracts/package.json, commit the corresponding lockfile entry, and import it through requireFromContracts.

Discussion

ducksquadd

<https://github.com/NestUSD/contracts/commit/735c8cf8072d9daba5262065af7db5736b70f3ee>

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.